Cloud Computing

Il bello di vivere

Vincenzo Salvatore

Raffaele Mercurio

Contents

1. Introduction

Cloud computing is essentially the evolution of computing into a public utility, much like electricity, water, or the telephone system. It is a model that allows consumers to access computing services on demand and pay only for what they use.

Key Word

Ubiquitous: server and data can be located anywhere and accessed from anywhere

This **cloud model** is composed of **five** essential characteristics, **three** service models, and **four** deployment models

1.1 Essential Characteristics of Cloud Computing (NIST)

1. **On-demand self-service:** users can automatically provision computing resources without human intervention from the service provider.
   - **Option A**: Fully atomated service provisioning
   - **Option B**: The provider use an automated interface to request and truck the service, but provider may use manual labor to provisioning the interface.
2. **Broad network access:** Possibility to access cloud resourcess from internet by using different devices.
   - **Option A**: Available over the internet
   - **Option B**: Available over a network that is available from all the access points the customer requires.

3. **Resource pooling:** Computing resources are pooled to serve multiple consumer using a **Multi-tenant model**.

   #### Multi-Tenant Model

   The computing infrastructure is shared among more than one customer, folowing this principles:
   - multi users (tenants) access the same application logic simultaneously.
   - Each tenant has its own view of the application as a dedicated istance.
   - Multitenant application isolate data and configuration for each tenant.
4. **Rapid elasticity:** Capability of the system to adapt to workload that rapidly changes.
   - **Option A**: Resource allocation is automated and near to real time.
   - **Option B**: Not fully automated, but fast enough to support the requirements.

   Two type of Scaling:
   - **Vertical Scaling:** increase the capacity of a single resource (e.g., adding more CPU or RAM to a server).
   - **Horizontal Scaling :** add more instances of a resource (e.g., adding more servers to a cluster).

   Example

   Black Friday is a prime example of a situation where rapid elasticity is crucial. During this shopping event, there is a significant surge in online traffic and demand for products. E-commerce platforms need to quickly scale their resources to handle the increased load and ensure a smooth shopping experience for customers. Cloud computing allows these platforms to rapidly allocate additional resources, such as servers and bandwidth, to accommodate the spike in traffic without any downtime or performance issues.
5. **Measured service:** Cloud service charateristics are strictly monitored with enough details in order to guarantee specific **Service Level Agreement (SLA)** and then charge customers for that services.

1.2 Service Models

The **service models** in cloud computing are designed to define the level of control a *consumer* has over the cloud resources and to clearly separate the responsibilities between the **cloud consumer (subscriber)** and the **cloud provider**. According to the *NIST framework*[**ahmad2023deep**], there are three primary service models:

1. **Infrastructure as a Service (IaaS):** The consumer has control over operating systems, storage, and deployed applications; and possibly limited control of select networking component.

   Example

   Examples include Amazon Web Services (AWS) EC2, Microsoft Azure Virtual Machines, and Google Cloud Compute Engine.
2. **Platform as a Service (PaaS):** The consumer has control over the deployed applications and possibly configuration settings for the application-hosting environment.

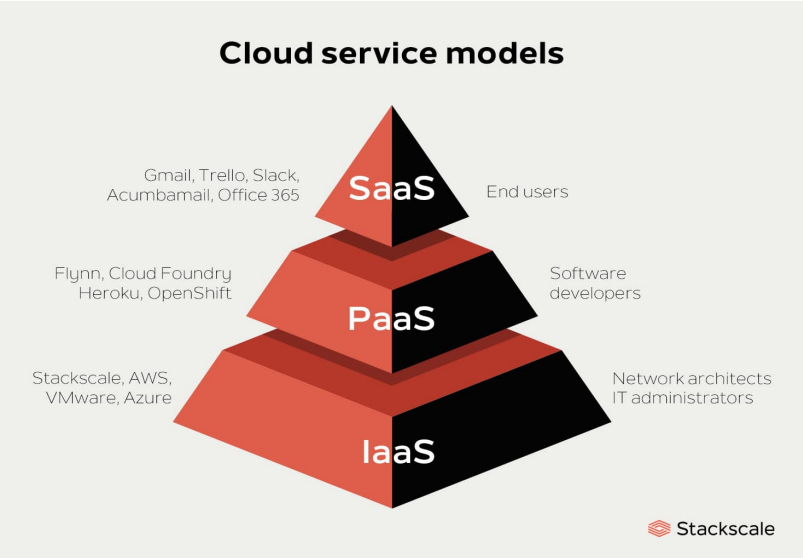
Example

Examples include Google App Engine, Microsoft Azure App Service, and Heroku

3. **Software as a Service (SaaS):** The cloud provider hosts applications and makes them available to consumers over the internet. Consumers can access and use the software without managing the underlying infrastructure or platform.

Example

Examples include Google Workspace (formerly G Suite), Microsoft Office 365, and Salesforce.



1.3 Deployment Models

While service models determine what you get, *deployment models* determine *how the infrastructure is hosted*, *who manages it*, and *who is allowed to access it*. According to the *NIST framework*[**ahmad2023deep**], there are four primary deployment models:

1. **Public Cloud:** This is the most open deployment model. The cloud infrastructure is provisioned for open use by the general public and is owned, managed, and operated by a cloud provider. As your notes enthusiastically summarize, it is *All you can access!!!!*
2. **Private Cloud:** Is dedicated entirely to a specific customer, ensuring logical and physical isolation from other users. This model offers maximum control and security.

Note

A closely related concept is the **Virtual Private Cloud (VPC)**. Solutions like Amazon VPC and Google Cloud VPC provide VPN features to securely connect your resources within a public cloud environment while providers like IBM and HP offer cloud environments with enhanced physical isolation.

3. **Hybrid Cloud:** This model is a combination of two or more distinct cloud infrastructures, such as mixing a private cloud with a public cloud.
4. **Community Cloud:** Listed as the fourth model by NIST, this is a cloud infrastructure shared by several organizations that support a specific community with shared concerns.

Example

Examples include government agencies sharing a cloud for public services or research institutions collaborating on a shared cloud for scientific research.

1.4 Evaluation of Cloud Computing Services - NIST 800-145

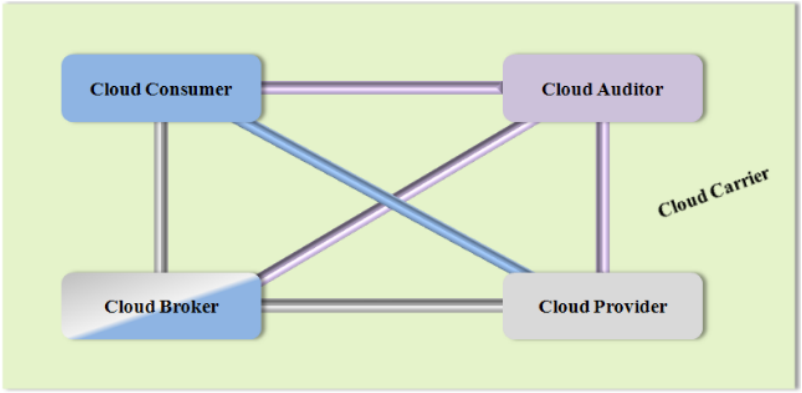
This section is used to determine whether a specific capability is actually a cloud service, allowing you to verify it through three specific tools:

1. **Cloud Service Worksheet:** to confirm if a service is a cloud service, just looking **Essential Characteristics of Cloud Computing??**.
2. **Cloud Service Model Worksheet:** to determine the service model of a cloud service, just looking **Service Models**1.2.
3. **Cloud Deployment Model Worksheet:** to determine the deployment model of a cloud service, just looking **Deployment Models**1.3.

1.5 The Cloud computing Reference Model - NIST SP 500-292

In this section, we will focus on **Cloud Roles**, the framework defines five major actors that interact within this cloud ecosystem:

1. **Cloud Consumer:** The individual or organization that uses the cloud services.
2. **Cloud Provider:** The entity responsible for making the cloud services available to consumers.
3. **Cloud Auditor:** An independent party that assesses the security and performance of the cloud services.
4. **Cloud Broker:** An intermediary that manages the use, performance, and delivery of cloud services, and negotiates relationships between cloud providers and consumers.
5. **Cloud Carrier:** The intermediary that provides connectivity and transport of cloud services from providers to consumers.



1.5.1 Example of Cloud Roles and their interactions

Cloud Broker

The **Cloud Broker** acts as a middleman between the consumer and various cloud providers. Instead of the consumer buying and managing services directly from multiple companies, the broker aggregates, integrates, or customizes these services into a single, cohesive package. It can provide services in three categories:

1. **Service Intermediation:** A Cloud Broker can enhance a service by adding value to it, such as providing additional security features, performance monitoring, or billing services.
2. **Service Aggregation:** A Cloud Broker can combine services from different providers into a unified offering, simplifying management for the consumer.
3. **Service Arbitrage:** A Cloud Broker can dynamically select services from different providers based on factors like cost, performance, or availability, optimizing the consumer's experience.

Example

Real-world example: Imagine a company that needs storage from Google Cloud, computing power from Amazon Web Services (AWS), and a database from Microsoft Azure. Instead of managing three separate accounts and bills, they hire a Cloud Broker (like Flexera or a specialized IT consultancy). The broker provides a single dashboard to manage everything, optimizing costs and handling the complex technical integration behind the scenes.

Cloud Auditor

The **Cloud Auditor** is an independent, third-party organization. Its job is to objectively evaluate the cloud provider's services, focusing heavily on security protocols, privacy standards, and performance metrics. This ensures the provider is actually delivering what they promised and adhering to legal regulations.

Example

Real-world example: A bank wants to move its sensitive financial data to a new Cloud Provider but needs to be absolutely certain the data won't be compromised. An independent Cloud Auditor (like Deloitte or a certified ISO 27001 inspector) runs a comprehensive security check on the provider's servers. They then issue an official certification report. The bank (Consumer) reviews this audit report to verify the Provider's reliability before signing the final contract.

1.6 Service Orchestration

In the NIST Reference Architecture, **Service Orchestration** is a core responsibility of the **Cloud Provider**. It refers to the composition, arrangement, coordination, and management of various system components and computing resources in order to successfully deliver cloud services to the **Cloud Consumer**. To make this work, the architecture uses a three-layered model that directly integrates the concepts we have discussed so far:

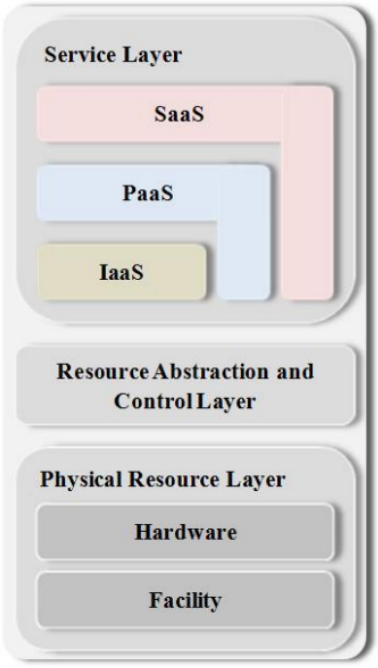


Figure 1.1: Service Orchestration in the NIST Reference Architecture

1. **Service Layer:** This is the interface layer where the Cloud Provider actually exposes the services to the Cloud Consumer. Depending on the model the consumer chooses, the orchestration tasks differ.

Example

For example:

- **IaaS**: Requires the orchestration of Virtual Machines (VMs) and virtual devices;
- **PaaS**: Often relies on container orchestration (like Kubernetes);
- **SaaS**: Relies on orchestrating microservices and Service-Oriented Architecture (SOA)

2. **Resource Abstraction Layer:** This is the "software fabric" that bridges the gap between the physical hardware and the services you buy. It uses software components like hypervisors to create virtual machines, virtual storage, and network abstractions.

Note

This layer is what makes the **Five Essential Characteristics** possible. The control software in this layer handles resource allocation and usage monitoring, which directly enables Resource Pooling, Rapid Elasticity, and Measured Service.

3. **Physical Resource Layer:** This foundational layer consists of all the actual physical equipment. It includes the hardware (CPUs, memory, network routers, firewalls, switches, and hard disks).

1.7 Cloud Service Management

Cloud Service Management is a core architectural component managed by the Cloud Provider that encompasses all the service-related functions necessary for the management and operation of the services required by, or proposed to, cloud consumers.

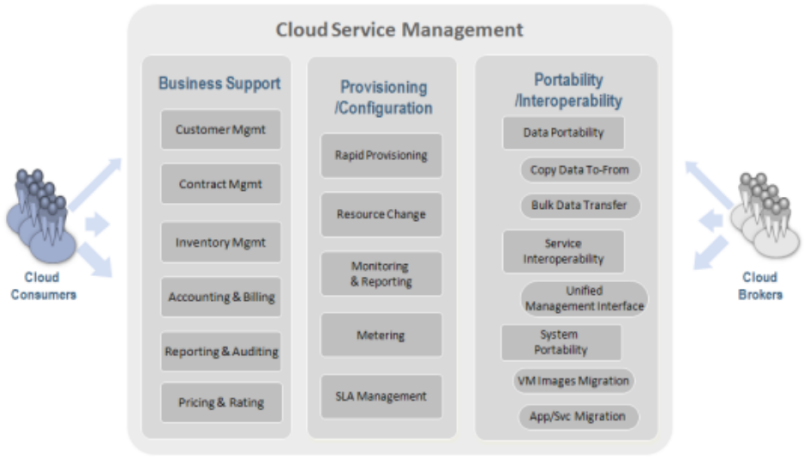


Figure 1.2: Cloud Service Management in the NIST Reference Architecture

According to the NIST Reference Architecture, is divided into three main operational perspectives:

1. **Business Support**: This area entails the set of business-related services dealing with client-facing operations and supporting processes.

2. **Provisioning and Configuration**: This category focuses on the technical delivery, monitoring, and lifecycle management of the cloud resources
3. **Portability and Interoperability**: Are crucial capabilities designed to prevent vendor lock-in (remaining p̈risoneröf only one cloud provider), and give consumers flexibility across multiple cloud environments.
    - **Portability:** The ability to move applications and data from one cloud provider to another without significant rework.
    - **Interoperability:** The ability of different cloud services and platforms to work together seamlessly, allowing for integration and communication across different cloud environments.

2. Enabling technologies

2.1 Distributed Systems

A **distributed system** is a collection of independent computers that appears to its users as a single coherent system.

2.1.1 Distributed Computing

Distributed computing, differently from the concept of distributed systems, is a **paradigm**.
Define **How the computation is carried on**. In a distributed architecture, a massive or complex task is not handled sequentially by a single machine. Instead, the computation is divided into smaller, manageable units that are executed concurrently (at the exact same time) This parallel execution is what allows distributed systems to process massive amounts of data much faster than a single computer ever could.

The computing elements

These concurrent units of computation can be assigned to various types of hardware setups

- Cores within the same processor
- Processors on the same computer
- Processors on different nodes

Different Locations

When we say a system is "distributed", it often implies that the physical computing elements are not located in the same place. They could be in different server racks within a single room, or spread across completely different data centers around the world. This geographical distribution is critical because it ensures **reliability**

and fault tolerance;
if one location loses power, the computation can continue elsewhere.

Heterogeneity (Hardware and Software)

This is one of the most powerful features of distributed computing. The nodes making up the system do not need to be identical clones of one another.

- **Hardware heterogeneity**: You can combine powerful, expensive servers with smaller, cheaper commodity hardware within the same network.
- **Software heterogeneity**: These machines can run entirely different operating systems or programming environments.

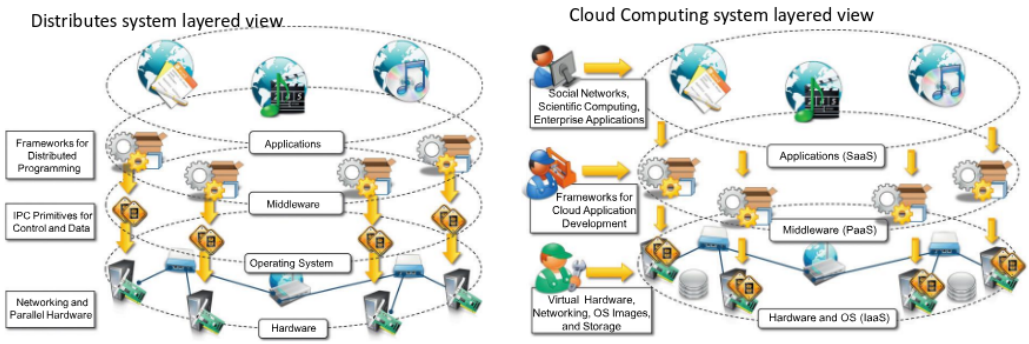


Figure 2.1: Cloud computing is a specialized form of distributed computing that introduce service model.

2.2 Components of a Distributed System

The components of a distributed system are:

- **Applications**;
- **Middleware**: is the part involved in the management of the lower layers in order to make look like a unique system;
- **Operating Systems**;
- **Hardware**.

Cloud Computing is a specialized form of ditributed system, where:

- Application $\equiv$ SaaS
- Middleware $\equiv$ PaaS
- OS and Hardware $\equiv$ IaaS

2.2.1 Architectural Style

An Architectural style defines the following concepts:

- the different roles of the components in the distributed system;
- how components are distributed across multiple machines;
- what communication mechanisms are used.

2.2.2 Roles of components

Components have their own life-cycle and interact each other to perform their activities. Each process provides services and can user services exposed by other processes. They are physical organized in architectures like client/server, peer-to-peer and service-oriented.

Event-Based Systems

In event-based systems each component publish a collection of events that the others components can subscribe to receive a callback when the event occurs.

Client-Server Archutecture Style

In client/server architecture style there is a client that sends a request and a server that elaborates and replies to it. This style is suitable for many-to-one scenarios.

There are three main components (presentation, application logic and data storage) and two possible models:

- **Thin-client model**: client has only the presentation and so the server does all the works (application logic and data storage);
- **Fat-client model**: client does also some application logic.

These three components can be seen as conceptual layers, called **tiers**:

- **Two-Tiers** (classic model): it's suitable only for systems of limited size since it suffers from scalability issues;
- **Three-Tiers** or **N-Tiers**: are more scalable because it's possible distribute the tiers into several computing units.

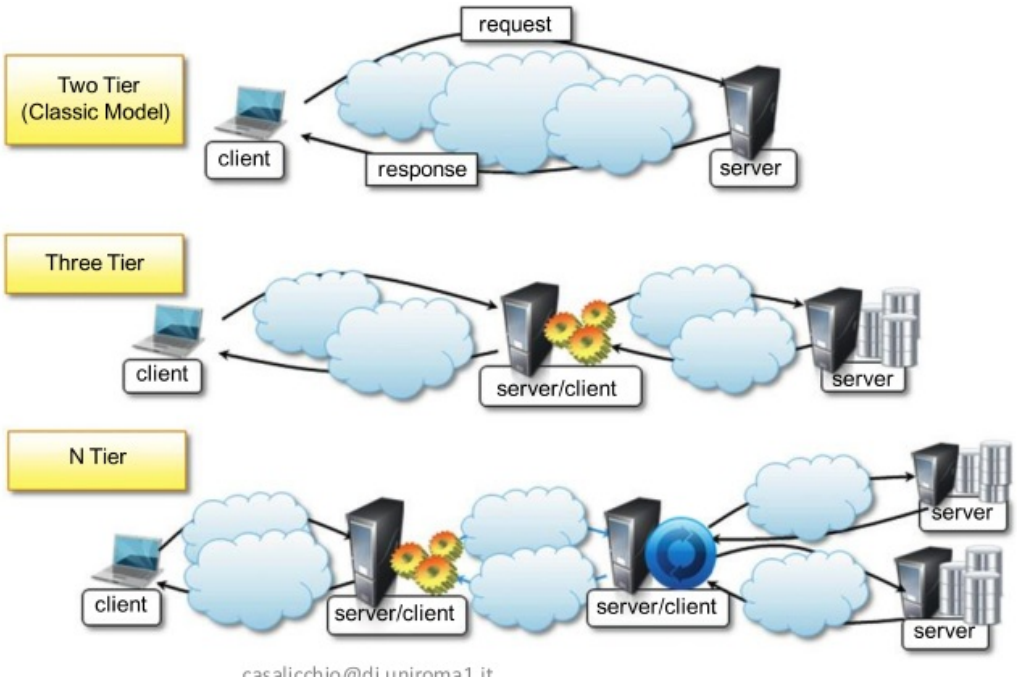


Figure 2.2: Difference between models with two or more tiers.

Peer-to-Peer

In the peer-to-peer architecture style there is simmetry since all the components play the same role and incorporate both client and server capabilities.
It's suitable for highly decentralized systems, like file sharing applications, distributed and cloud storage systems.

Service Oriented Architectures (SOA)

The Service Oriented Architecture is an architectural style that supports service orientation.

2.3 Service Orientation

A service:

- is a logical representation of a repeatable activity that has a specified outcome (like provisioning of weather data or checking customer credit);
- is self-contained (autonomous);
- may be composed of other services;
- seems as a black box to consumers of the service.

2.3.1 Charateristics of Services

An application is an aggregation of services that are coordinated within a SOA. Services have four main charateristics:

- **Explicit Boundaries**: interfaces are kept minimal in order to make simpler reusing them;
- **Autonomy**: services can be integrated in different systems at the same time and can handle failures;
- **Share of schema and contracts**: a contract describes the structure and the order of messages the services can receive;
- **Compatibility based on policy**: the policies describe the requirements of the services.

2.3.2 Major Roles

There are three major roles within SOA:

- **Service Provider**: creates the service and publishes it on a registry;
- **Service Consumer**: locates the service in the registry and use it;
- **Service Registry**: store the service descriptors.

The iter is:

1. the consumer discovers a service on the registry;
2. the registry returns binding informations;
3. the consumer establishes a connection with the provider using the binding informations;
4. the consumer can use the service.

2.3.3 Benefits of SOA

The most relevant benefits of a service-oriented architecture are:

- service re-use;
- message monitoring, control and transformation;

- complex event processing;
- service composition;
- service discovery.

2.4 Orchestration and Choreography

In order to coordinate and manage services, we can use:

- **Service Orchestration**: it coordinates the composition and interaction of services in a centralized way. The orchestrator is responsible for involving and combining the services.

Example

An example of orchestration is the travel planner where a centralized unit reuses all the services, like select destination, display hotels, select hotels, ecc...

- **Service Choreography**: it coordinates the interaction of services without a single point of control. There are some rule of interactions and the coordination happens through the exchange of messages between two or more services.

Example

An example of choreography is the authentication of a system where the client service sends a request to the Auth service, which will check the credentials and will send a request for three factor authentication and so on.

2.5 Microservices

A microservice architecture is the style that structures an application as a collection of services that are:

- highly maintainable and testable;
- independently deployable;
- owned by a small team;
- organized around business capabilities;
- loosely coupled.

This architecture enables the rapid and frequent release of complex applications.

2.5.1 Scalability

Microservices create a new dimension of scalability:

- **X-axis** scaling (horizontal duplication): from one instance to more instances;
- **Y-axis** scaling (functional decomposition): from a monolith to microservices;
- **Z-axis** scaling (data partitioning): from one partition to many partitions;

2.5.2 Monolith Features

A monolithic architecture has some pros:

- it's simple to develop and to drastically change;
- it's easy to scale horizontally (duplication)

But it has also some contra:

- when the complexity increase, it's more difficult to scale.

2.5.3 Monolith vs. Microservices

In the microservice architecture there is an API Gateway to route the requests to different microservices, each one with different databases.
Even the production path is different between these two architecture styles:

- in a monolith, a huge team works on a single code base, creating queues and bottlenecks because of complex testing;
- in microservices, small teams work on separate repositories with automated deployment pipelines, making everything faster and more reliable.

2.5.4 SOA vs. Microservices

SOA and microservices differs about various aspects:

- **Taxonomy**: SOA has many abstract layers (business, middleware, enterpris, application and infrastructure) and requires a high level of coordination between different teams to handle a single request. Microservices have only APIs layer and functional/infrastructure services. So a single team manage all the service without any external coordination.

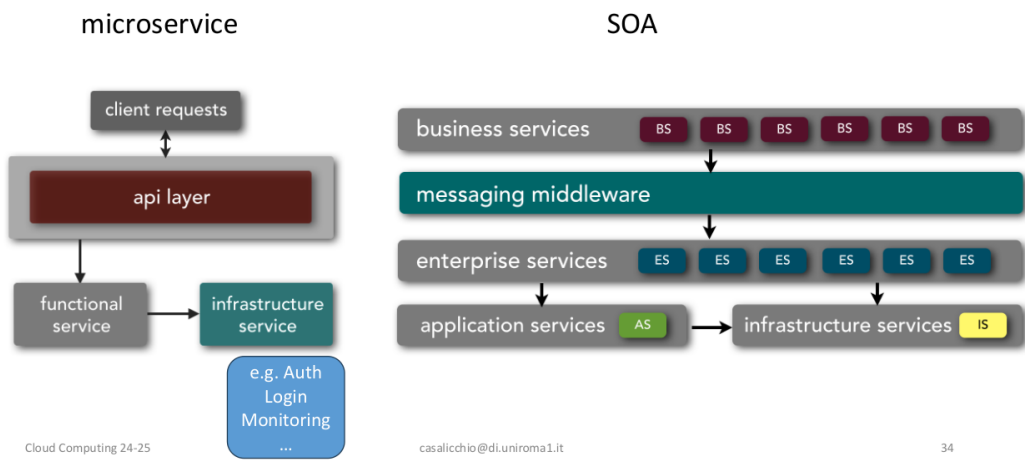


Figure 2.3: Difference of taxonomy between SOA and microservices.

- **Service Ownership and Coordination**: SOA requires coordination while microservices require minimal coordination.
- **Service Granularity**: microservices are single-purpose (specialized) services while services within SOA can range their size from an application service to very large enterprise services.
- **Component Sharing**: SOA follows the share-as-much-as-possible architectural style but this can lead to high connection between components and so there can be more risk associated with changes. Microservices follow the share-as-little-as-possible architectural style, which uses the *bounded context*. This means that all the components and their data represent a closed entity with minimal dependencies.

This style could violate the don't-repeat-yourself principle but it makes maintainance and deployment drastically easier and safer.

- **Service Choreography and Orchestration**: microservices prefer choreography since they don't have a central middleware component. Too much choreography can lead to *efferent coupling*, meaning that if a service depends on different services, the system becomes too rigid.
  SOA relies on both orchestrations and choreography.
- **Middleware and API Layer**: SOA relies on messaging middleware to coordinate service calls and translate protocols between incompatible services.
  Microservices support the notion of API layer that acts as a service-access interface.
- **Application Scope**: it refers to the overall size of the application that an architecture pattern can support.
  SOA is well-suited for large and complex systems that require integration with many heterogeneous applications and services.
  Microservices architecture is better-suited for smaller, well-partitioned systems.
- **Heterogeneous Interoperability**: SOA accepts every protocol through its middleware and can integrate systems implemented in different programming languages and platforms.
  Microservices attempts to simplify the architecture by reducing the number of choices for service integration.

3. Containerization

3.1 Containerization

Containerization is different from virtualization. In the latter we have a hypervisor that manage the communication and the relationship between the system and the VMs that can be hosted. In containerization, instead, we have a unique OS, shared with all the containers.
A container can be used to distribute and run a single application (or a microservice).
Containerization is the evolution of *chroot*, the operation that change the root of the file system for a process in order to isolate it. In this way the process (and its hcildren) cannot have access to other partitions of the file system.

3.1.1 Foundations of Containerization

Containerization is based on:

- **Namespace**: it has the goal of abstract the global system resources (such as PIDs, network devices, mount points, ecc...) in order to give the perception to the processes of being in their own isolated instance of the system.
  Changes to the resources in the namespace are visible only by its members and invisible to other processes.
  We can create one or more namespaces by creating a process using the clone() system call.
  We can assign a process to an existing namespace using setns().
  We can move a process to a new namespace using unshare().

- **Cgroups**: its main goal is to support resource limiting, prioritization, accounting and controlling. It allows to organize processes into hyerarchical groups.
  The kernel provides a cgroup interface through a pseudo-filesystem called cgroupfs.
  There exists different controller to control different resources (i.e. one for the CPU, one for the memory, one for the PID, ecc...).
- **UnionFS**: it allows to keep files separated physically, but merging them logically into a single view.
  Each physical directory is called branch and each branch has a different priority. For example if there exist two file with the same name in different branches then in the merge we will have only the one in the branch with largest priority. If there are two directories in different branches but with the same name, in the merge we will see a union of them.
  UnionFS allows to mix read-only and read-write branches. In this case the union is read-write, since the copy-on-write mechanism gives the illusion that you can modify files and directories on read-only branches while keeping the base intact.

3.2 Docker

Containers run directly on the host's kernel, wheter the host is physical or virtual.
The **Docker engine** is a client/server application, where the client is the Command Line Interface (CLI), through which the user can interact, and the server is the Docker daemon.
The **communication** is done through the REST API, that can be also called separately (called from the application).
The server (**daemon**) is the one responsible of creating and managing physically the Docker objects (like networks, containers, images and data volumes).
From the **client** some commands (like docker build, docker pull and docker run) are launched.
The **images** are the templates (read-only) that contains the instructions used to create containers. There is also a registry, where the daemon can download the images.
Containers are simply runnable instances of an image.
An example of a Docker CLI command id $docker run -i -t ubuntu/bin/bash$, which:

- locate and eventually download the ubuntu image (if not locally stored);
- create a new container;
- allocare a read/write file system as final layer on the image;
- isolate the file system created from the host file system;
- create a network interface connected to the default network;
- start the container and run the /bin/bash command.

3.2.1 Storage Options

In Docker, data management is critical because containers are inherently **ephemeral**: if a container is deleted, any data written inside it is lost. To manage persistence and storage, Docker offers different options.

- **Container Writable Layer**
  When you create a container, Docker adds a thin, temporary writable layer (the "container layer") on top of the underlying read-only image layers.

  - *How it works:* All changes, new files, or deleted files during container runtime are saved here. Docker uses a *Storage Driver* (e.g., `overlay2`) with a *Copy-on-Write* strategy.
  - *Persistence:* **No.** If the container is deleted (`docker rm`), this layer is permanently destroyed.
  - *Performance:* Lower compared to the host's native file system due to the storage driver overhead.
  - *Best for:* Temporary files specific to the container's lifecycle that have no value once the container is stopped (e.g., volatile session logs).

- **Volumes**

  Volumes are the **recommended mechanism** by Docker for persisting data. They are completely managed by Docker and reside in an isolated part of the host file system (`/var/lib/docker/volumes/` on Linux).

  - *How it works:* Docker creates a dedicated directory on the host and mounts it inside the container. Non-Docker processes on the host should not modify this directory.
  - *Persistence:* **Yes.** If you delete the container, the volume remains intact and can be attached to new containers.
  - *Performance:* Excellent, since data bypasses the storage driver and writes directly to the host's native file system.
  - *Extra Advantages:* They support volume drivers (to store data on cloud providers like AWS S3 or NFS network shares) and are easy to back up or migrate.
  - *Best for:* Databases (e.g., MySQL, PostgreSQL), production storage, and safely sharing data among multiple containers.

- **Bind Mounts**

  Bind mounts link **any specific file or directory** from your host machine into the container.

  - *How it works:* Docker does not manage this directory; you point to an absolute path on the host system (e.g., `/home/user/projects/app`). If the host directory contains files, they temporarily overwrite the container's target directory content.
  - *Persistence:* **Yes.** Any modification made inside the container is instantly reflected on the host, and vice versa.
  - *Performance:* Maximum, equal to the native file system performance.
  - *Disadvantages:* Heavily reliant on the host OS directory structure (moving the container to a different OS or server might break it). It also exposes the host to potential security risks if the container runs with root privileges.
  - *Best for:* **Software development.** You can mount your source code into the container: editing code on your host IDE (e.g., VS Code) immediately updates the running container app via hot-reloading.

- **tmpfs Mounts**

  Unlike volumes and bind mounts, a `tmpfs` mount is **entirely in the host's memory (RAM)**. Data is never written to disk.

  - *How it works:* Docker reserves a portion of the host's RAM and mounts it inside the container.
  - *Persistence:* **No.** When the container stops, the `tmpfs` data is wiped out instantly. It cannot be shared between different containers.
  - *Performance:* Extremely high (RAM speeds).
  - *Best for:* Highly sensitive data that you do not want written to disk (e.g., encryption keys,

temporary passwords) or high-frequency state files that would wear down disk I/O (e.g., volatile caches).

Table 3.1: Comparison of Docker Storage Options

Feature	**Writable Layer**	**Volume**	**Bind Mount**	**tmpfs**
Data Location	Container top layer	Docker Area (Host)	Any Folder (Host)	System RAM
Persistent?	No	**Yes**	**Yes**	No
Docker Managed?	Yes	**Yes**	No	Yes
Performance	Reduced	High (Native)	High (Native)	**Maximum (RAM)**
Typical Use Case	Temporary logs	Prod Databases	Dev code sharing	Sensitive/Cache data

Example

For example:

1. **Volumes** can be used in these situations:
   (a) to backup, restore or migrate data from a Docker host to an other;
   (b) to share data among multiple running containers;
   (c) no guarantee to have a given directory or file structure in the Docker host;
   (d) to store container's data on a remote host or a cloud provider, rather than locally.
2. **Bind mount** can be used in these situations:
   (a) to share configuration files from the host machine to the containers;
   (b) to share source code or to build artifacts between a development environment on the Docker host and a container;
   (c) the file or directory structure of the Docker host is guaranteed to be consistent with the one the container require.
3. **Tmpfs** can be used in these situations:
   (a) data should not persist either on the host machine or within the container for security reasons or to manipulate large volumes of non-persistant data with high performance.

3.2.2 Networking

Docker isolates containers from a networking perspective. By default, a container cannot freely communicate with the external world or other containers unless it is placed into a specific network. Docker offers 5 native network drivers.

- **Bridge Network**
  The `bridge` driver is the **default network** in Docker. If you do not specify a network when launching a container, it connects to the default bridge.
  - *How it works:* Docker creates a software-based bridge (usually called `docker0`) on the host OS. Connected containers receive a private internal IP address (e.g., `172.17.0.x`) and can communicate with each other using these IPs. External access is achieved via NAT (Network Address Translation).
  - *Isolation:* High from the host, but containers on the same bridge can communicate.
  - *DNS Resolution:* If you create a *user-defined* bridge network (`docker network create`), Docker enables an automatic internal DNS. Containers can communicate using just the **con-**

tainer name, regardless of IP changes.
  - *Best for:* Standard applications running on a single host (e.g., a backend container and a database container that need to talk to each other on the same server).

- **Host Network**
  With the `host` driver, the network isolation between the container and the host machine is **completely removed**.
  - *How it works:* The container does not get its own IP address. It directly shares the network interface and IP of the host machine. If a container starts a service on port `80`, that service is instantly accessible on port `80` of the host's IP.
  - *Performance:* Maximum. There is no overhead from NAT or Docker routing; network packets travel at native host speed.
  - *Disadvantages:* Total lack of isolation and potential port conflicts. You cannot run two containers on the `host` network listening on port `80` simultaneously.
  - *Best for:* Applications requiring extreme network performance or managing an enormous range of ports (e.g., media streaming servers, WebRTC, or monitoring tools).
- **Overlay Network**
  The `overlay` driver connects containers running across **different physical host machines (or VMs)**.
  - *How it works:* It creates a distributed virtual network on top (*overlay*) of the physical networks of the hosts. It is the native network used by **Docker Swarm**. It utilizes an encrypted tunnel (typically VXLAN) to make containers behave as if they were on the same local network, even if the servers sit in different data centers. *Isolation:* Secure and encrypted. Eliminates the need for OS-level routing between various servers.
  - *Best for:* Microservices architectures distributed across multiple servers, production clusters, and Docker Swarm stacks.
- **Macvlan Network**
  The `macvlan` driver allows you to assign a **unique physical MAC address** to each container, making it appear on the local network as if it were a physical device (like another independent PC or server).
  - *How it works:* The container bypasses the Docker network stack entirely and connects directly to the host's physical network interface. It receives an IP directly from your local router or DHCP server.
  - *Disadvantages:* Complex to configure. It requires the host's network card to support "promiscuous mode". Note: by default, the host cannot communicate directly with its own `macvlan` containers without specific routing workarounds.
  - *Best for:* Legacy applications that expect to be integrated directly into a physical network, or when performing actual network traffic monitoring (e.g., Intrusion Detection Systems).
- **None**
  The `none` driver **completely disables networking** for the container.
  - *How it works:* The container starts in absolute isolation. It only has access to the local loopback interface (`localhost` / `127.0.0.1`), but lacks any virtual network card pointing to other containers or the outside world.
  - *Best for:* Containers performing purely computational or highly sensitive batch jobs (e.g., compil-

ing heavy reports, generating cryptographic keys, parsing local files) where you need absolute certainty that no data can enter or leave via the network.

Table 3.2: Comparison of Docker Network Drivers

Driver	Scope	Container IP	Performance	Host Comms	Main Use Case
Bridge	Single Host	Private Virtual IP	Good (with NAT)	Via exposed ports	Standard multi-container app
Host	Single Host	Same as Host IP	**Maximum**	Shares host stack	High perf / WebRTC apps
Overlay	Multi-Host	Cluster Virtual IP	Medium (Encap.)	Orchestrator managed	Distributed Swarm / Microse
Macvlan	Physical LAN	Real LAN IP	High (Bypasses Host)	No (Isolated by def.)	Legacy apps / Router integrat
None	Isolation	None (loopback)	N/A	**None**	Isolated batch tasks / High se

Example

For example:

1. **Bridges** can be used when multiple containers should communicate on the same Docker host;
2. **Host** network can be used when the network stack should not be isolated from the Docker host;
3. **Overlay** network can be used when containers are running on different Docker hosts and they should communicate;
4. **Macvlan** network can be used when you are migrating from a VM setup or when you need your containers to look like physical hosts on the network.

4. Virtualization

4.1 Virtualization

The virtualization environment consists of a virtualization layer, with an hypervisor or virtual machine monitor (VMM), between the physical hardware (host) and the virtual machine (client)

4.1.1 Main Features

Virtualization guarantees three main charateristics:

- **Increased security**: the VMM controls and filters the activity of the guest, thus preventing some harmful operations from being performed;
- **Managed execution**: the physical resources are converted into virtual resources through some virtualization techniques like sharing, aggregation, emulation and isolation;
- **Portability**: it's possible to transfer and use data and applications form a computing platform to an other. A virtual machine is booted from a VM image that is shared in a specific disk format. You can move around VMs keeping or converting this format.

4.1.2 Layered Structure

A computer system is characterized by a layered structure where the levels can communicate with each other through some interfaces.

4.1.3 Privilege of Instructions

The processor instructions can be divided in:

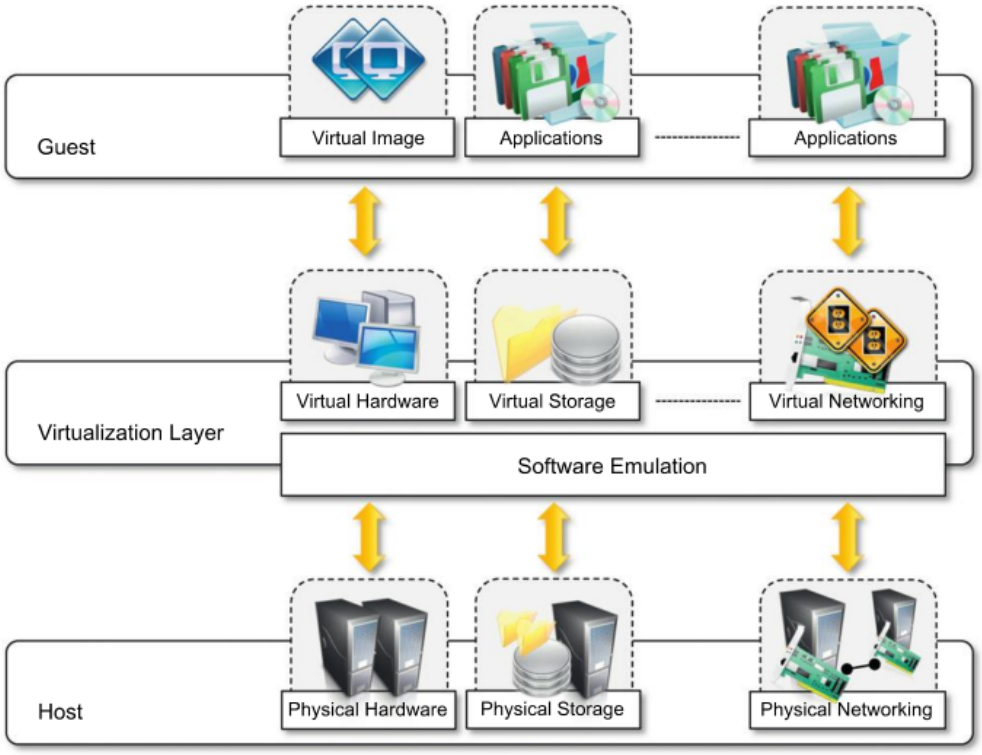


Figure 4.1: Virtualization model

- **Non-privileged**: this one can be used without interfering with other tasks. An example can be arithmetic instructions This ring of instructions is called User mode;
- **Privileged**: this one are executed under specific restrictions and are mostly used for sensitive operations.

The VMM run in supervisor mode (the most privileged instructions).

4.2 Virtual Machine Manager and Hypervisor

4.2.1 Level of Virtualization

There can be two level of virtualization:

- **Type-II hypervisor**: the hypervisor uses system calls in order to communicate with the operative system;
- **Type-I hypervisor**: the hypervisor uses ISA calls to communicate directly with the hardware.

4.2.2 Properties to Satisfy

In order to create a VMM, three properties must be satisfied:

- **Equivalence**: a guest running under the control of a VMM should exhibit the same behaviour as when it's executed directly on a physical host;

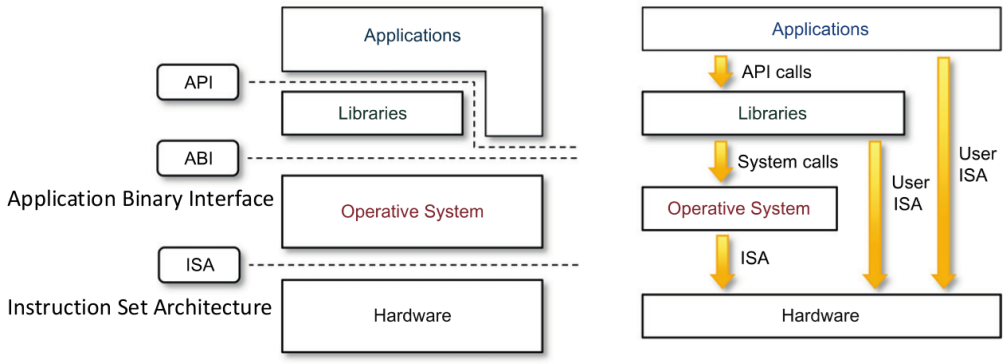


Figure 4.2: Layered structure of a computer system

- **Resource Control**: the VMM should be in complete control of the virtualized resources;
- **Efficiency**: a statistically dominant fraction of the machine instructions should be executed without any intervention of the VMM.

4.2.3 Composition

The VMM consists o three main modules:

- **Dispatcher**: it's the entry point of the virtual machine instance requests that are routed to one of the two other modules;
- **Allocator**: it's responsible for deciding the system resources to be provided to the VM. If the VM instance wants to change the resources allocated to it, the allocator is invocked by the dispatcher;
- **Interpreter**: it's composed of routines that are activated everytime the VM tries to execute a privileged instructions. In this case a trap is executed and the corresponding routine is triggered. In

The Type I hypervisor is installed directly on the hardware. In this case the VMM can be a microkernel on a monolithic architecture.

The trap of the interpreter is triggered when a guest tries to execute an instructions that is reserved to the kernel mode. In this case the hypervisor will control the instruction:

- if the instruction comes from a guest OS then it will execute it safely;
- if the instruction comes from a user program then it will emulate the behaviour of the hardware with such an instruction.

4.3 Full Virtualization

In **Full Virtualization** the OS does not know to be virtualized and only the privileged instructions are intercepted and emulated, while the other ones are passed directly to the hardware, without using the VMM.

4.3.1 Problem of Virtualization in x86 OS

A problem of virtualization was detected on x86 architecture when the sensible instructions don't coincide with the privileged ones. This made difficult to use the trap mechanism.

In order to solve this problem we can use the Binary Translation, which consists of searching for sensible

instructions by analyzing block by block the OS code. When a sensible instruction is found then it will be replaced with a safe call to the hypervisor.
A basic block is a short, straight-line sequence of instructions that ends with a branch (no jum, call, trap, return or any flow alteration).
This dynamic binary translation is not so expensive since most code blocks don't contain sensitive instructions and the other one are cached after be translated such that if the hypervisor will find an equal block later it will not have to translate it newly.
In 2005 Intel and then AMD have introduced specific instructions for virtualization allowing the hardware itself to handle traps natively. However, a lot of trap can paradoxally slow down the performance compared to an excellent binary translation.

4.4 Paravirtualization

In **Paravirtualization** the OS knows to be on a VM and so it directly substitute critical interactions with hypercalls using an intelligent compiler. So hypervisor expose an API that the guest OS uses rather than syscalls. This system (used for example by KVM and Hyper-V) is much more lite and avoids performance losses.
Paravirtualization uses the microkernel architecture. This microkernel handles base elements, like processes, interrupts and scheduling. While in full virtualization librarie communicate with an emulated hardware (causing traps), in paravirtualization we use specific libraries that directly launch hypercalls, extremely simplifying the process.

5. Automation & Orchestration

5.1 Automation vs Autonomic Computing

There is a critical difference between autonomic computing and automation:

- **Autonomic Computing** has the goal of keep the humans out of the loop, letting them only to provide high level objectives;
- **Automation** has the goal of reducing the human interaction and the manual processes

The Automation of IT operations works with software tools and technologies in order to automate organizational and operative processes of IT.
It automates repetitive tasks, orchestrates workflow and integrates automation into existing systems and applications.
The main benefits are:

- **Improved efficiency and productivity**: this is possible because we reduce the time and the effort of manual tasks. This free time of the IT staff can be used to focus on more strategic activities;
- **Reduced human errors**: we eliminate the risk of mistakes caused by manual interventions;
- **Saved time and costs**: we minimize the time wasted for repetitive tasks.

5.1.1 Data Centers Automation

Data centers have specialized platforms to automate key operations like provisioning, configuration, patching and monitoring.

Ansible

For example, Ansible is an open source tool used to manage configuration, deploy and orchestration.
It has two main features:

- **Agentless**: this means that no software is required on the remote machines in order to make them manageable.
  This eliminates overhead when the management is not in progress, making this tool ideal for high security and high performant environments.
  The instructions are prepared on the central server and then pushed to the hosts.
- **Push Model**: this means that Ansible manages remote machines by using frameworks that already exist on the platform (like SSH or WinRM (Windows Remote Management)).
  These framework are used to push the needed code (called modules) on the remote machines.
  The remote machines cannot see or affect how the other machines are configured.

The problem is that not all the nodes can run code (like the network devices). In order to automate such systems the module is executed locally on the Ansible control node.
On Linux or Windows nodes the module is copied and executed on the remote node and then deleted.

Ansible Playbooks

Ansible Playbooks use the YAML syntax to allow easy configuration across multiple machines. In this way they are repeatable and reusable. They can:

- declare configurations;
- orchestrate manual processes in a defined order on multiple sets of machines;
- launch tasks synchronously or asynchronously;
- define the system's desired state;
- be idempotent (if applied more than once, the state will not change).

The Playbooks consist of a series of **plays** (each one consists of multiple tasks) that define some automations to apply on a set of hosts (called **inventory**).
A task is a call to an Ansible module that is a small code for doing something that can be simple (like insert a configuration file) or complex (like create entire cloud infrastructures).
The core modules are written in such a way to allow an easy configuration of the desired state. The module checks if the action is actually needed before executing it. For example, if the task is defined to start a webserver then it will be executed only i fthe webserver is down.

Example

We can consider a three-tier web application. Its environment consists of:

- Applications servers;
- Database servers;
- Content servers;
- Load balancers;
- Monitoring system connected to an alert system (such as a notification service).

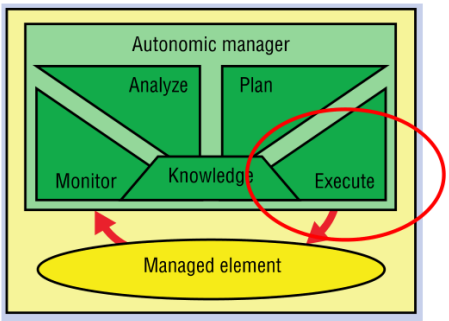
In this scenario Ansible can easily be used to orchestrate a rolling update without interventions. This process would consist of:

1. Consulting a cofiguration repository to get informations on the involved servers;
2. Configuring the base OS on all the machines and enforcing the desired state;
3. Signaling the monitoring system that the servers wil go down for a certain time;
4. Signaling the load balancer to not consider that servers for the load balancing;
5. Stopping the web application servers.
6. Deploying or updating the web application servers code, data and content;
7. Starting the web application servers;
8. Signaling the load balancer that it can newly consider that servers for the load balancing;
9. Signaling the monitoring to resume alerts on that servers;
10. Repeating this process for continous updatings;
11. Sending email reports and logginc as desired.

MAPE

Ansible represents the execute phase in the MAPE pardigm and it has the four self-* properties:

- **Self-Configuration**: when decided the reconfiguration, a playbook can be used to deploy the new system configuration;
- **Self-Optimization**: when decided for adding a new server, a playbook can do all the required actions;
- **Self-Healing**: if a server is degrading, a playbook that detach and reboot the server can be executed;
- **Self-Protection**: if a system is under attack because of vulnerabilities, a playbook for rolling an update can be deployed.



5.2 Orchestration

While automation involves setting up a single task to run autonomously, **Orchestration** involves automating a series of individual tasks so they work together as a single process or workflow.
The orchestration concept could vary depending on the technological context:

- service orchestration;
- IT infrastructure orchestration;
- container orchestration.

5.3 Container Orchestration

The container orchestration allows cloud and application providers to define how to select, deploy, monitor and dynamically control the configuration of multicontainer applications in the cloud.
It handles the deploy, run and mantaining phase, usually offering these main features: resource limit control, scheduling, load balancing, health check, fault-tolerance and auto-scaling.

5.3.1 Kubernetes

Kubernetes (**K8s**) is a platform for managing containerized workloads and services. This means:

- Service discovery and load balancing;
- Storage orchestration (automatically mounting the storage);
- Automated rollout and rollback (automatically bring the deployed containers to the desired state);
- Automatic bin packing (fit containers onto cluster nodes to make best use of the resources but giving them the needed CPU and memory);
- Self-healing (restart, replace and remove faulty containers);
- Secret and configuration management;

Main Concepts

There are three main concepts in Kubernetes:

- **Object**: they are persistent entities used to represent the state of the cluster. They can describe:
  - which containerized applications are running and on which nodes;
  - the available resources for those applications;
  - the policies (for example restart policies, upgrades and fault-tolerance).

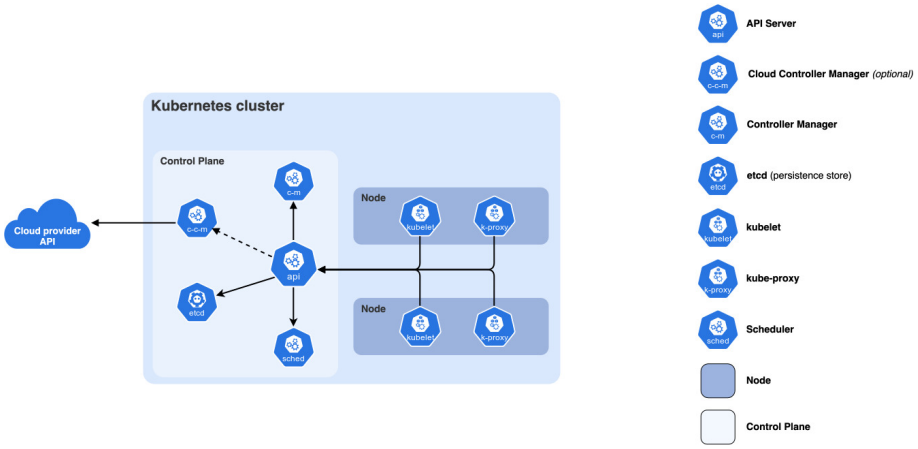
  A K8s object is a "record of intent": this means that once the object is created, the system will constantly work to ensure that the object exists.
  An object contains two main fields:
  - **Spec**: describe the desired state (for example I want 3 instances of the application);
  - **Status**: describe the current state of the object (for example there are 2 instances currently up and running);

Example

For example, if you create a deployment object with a spec of 3 replicas, the system reads the spec and start the 3 instances and updates the status. If an instance fails (status changes), K8s responds to the difference from the spec and the status by starting a replacing instance.

- **Pod**: it's the smallest deployable unit of computing that can be created and managed in K8s.
  A pod models an application-specific "logical host" that contains one or more containers: these containers are tightly coupled, share storage and network resources and the same specification for running themselves.
  A pod is similar to a set of containers with shared context (Linux namespace and cgroup) and shared file system.
- **Node**: they are the single virtual or physical machine, member of the K8s cluster or physical.
  Each node is managed by the control plane and contains the services necessary to run the pods.
  Nodes can enter a cluster through self-registration using the kubelet or through a manual add by the user.
  The node status is described by:
  - **address**: hostname and IP;
  - **conditions**: contains parameters like ready, disk pressure, memory pressure, PID pressure and network unavailable;
  - **capacity**: available resources like CPU, memory and maximum number of pods that can be scheduled onto the node;
  - **allocable**: amount of resources that are available to be consumed by normal pods;
  - **info**: general informations like kernel version, K8s version or OS;

K8s Cluster Components



A Kubernetes (K8s) cluster can be divided into two main components:

- **Control Plane**: is the brain of the system and runs on a dedicated server or on a VM. Its main components are:
  - **kube-apiserver**: is the frontend of the control plane. It scales horizontally;
  - **etcd**: is a key-value database which is highly available for sharing the cluster data;
  - **kube-scheduler**: it's the default scheduler in K8s and selects the optimal node for new pods based on resources requirements, hardware/software constraints and policies. Nodes that meet the scheduling requirements for a pod are called *feasible nodes*. The scheduler finds the feasible nodes (filtering, with PodFitsResources), give them a score (scoring, using some scoring rules), picks the one with the highest score (if there are many with equal score it will choose one at random) and

then notifies the API server about its decision (in a process called binding). If there are no feasible nodes, the pod will remain unscheduled until the scheduler will be able to place it;

  - **kube-controller-manager**: runs control processes like the node controller (to detect non active nodes) or job controller (to execute task until completion). It launches control loops that watch the state of a K8s cluster and make or request changes where needed. A controller tracks at least one K8s resource type comparing the spec (desired state) with the current state in order to try to match them via messages to the server API;

Example

To make an example we can consider a job controller. The job is the resource that start one or more pods in order to complete a task and then stop. When the job controller sees a new job, it makes sure that the nodes of the cluster are actually executing the correct number of pods to do that work. The controller, during the task, tells the API server to create or remove pods and update the task state. Once the task is completed, the job controller mark it as *finished*.

Example

We can also consider a controller that monitor external cluster resources, like the Cluster Autoscaler that makes sure that there are enough nodes in the cluster. If not, it is able to horizontally scale the nodes. This controllers read the desired state from the API server, but then communicate directly with an external system to scale the cluster in order to align to the desired state.

  - **cloud-control-manager**: links the K8s cluster to the API of a specific cloud provider.

- **Nodes**: they include the following components:
  - **kubelet**: is the agent running on each node in order to control that the containers described in a podspec are running and healthy;
  - **kube-proxy**: is a network proxy running on each node to manage and mantain network rules to allow communication inside and outside the cluster;
  - **container-runtime**: is the software responsible for running containers (for example Docker).

Node Affinity

Node Affinity in K8s is a set of rules used to specify preferences about how pods have to be placed on the nodes. It allows to constrain the pod execution on nodes that have specific labels.
Node affinity is useful for:

- ensuring compliance with data sovereignty laws.

Example

For example, if laws impose that data must be in Europe then node affinity force pods to run only on European nodes.

- optimizing network latency for distributed systems.

Example

For example, if two pods have to communicate a lot then we can force them to run on close nodes.

- allocating resources for high performance computing (HPC).

Example

For example, if an application needs high computation, we can force the pod to run on nodes with GPUs.

- handling workloads with specific storage requirements.

Example

For example if we need a database with quick queries, we can force it to run on nodes with SSD.

- supporting multi-tenancy and resource isolation.

Example

For example we can dedicate specific nodes to a specific enterprise.

Node Anti-Affinity is used to sepcify rules that prevent the scheduling of pods in the same nodes. It's particularly useful for highly availability (distributing pods across different nodes or zones) and for separating workloads for security or performance reasons.

6. Cloud Storage

6.1 Basic Concepts

6.1.1 Atomicity

An **atomic transaction** is a multi-step operation that should be complete without any interruption.
Atomicity requires non-interruptable ISA operations, like:

- **test-and-set**: writes to a memory location and returns the old content that has been replaced;
- **compare-and-swap**: compares the content of a memory location with a given value and if they are equals, then modifies the content of that memory location with a new value.

Atomicity also need mechanisms to access shared resources and to manage critical section for example with locks, semaphore or monitors.
There are two types of atomicity:

- **All-or-nothing**: it consists of the following phases:
  1. **pre-commit phase**: preparatory actions are done but can be undone.

     Example
     For example allocation of a resource or fetching a page from a secondary memory;

  2. **transition**: is the commit-point;
  3. **post-commit phase**: irreversible actions are done. For example write on a file.

  In this kind of atomicity, maintaining a history of changes and a log of actions allows to deal with system failures and consistency.

- **Before-or-after**: the result of every read/write is the same both if it occurrs completely before or completely after any other read/write.

Example

For example if we have a sequence of read and wirte operations in a transaction, then the concurrent execution should give the same result if we execute them subsequently.

6.1.2 Storage Model

A storage model is the layout of a data structure in physical storage. It's desiderable to have these properties:

- **read/write coherence**;
- **before-or-after atomicity**;
- **all-or nothing atomicity**.

The following are the two storage models:

- **Cell Storage Model**: it's based on the assuption that the memory is organized in cells of the same size and each object fits exactly in one cell.
  It reflects the physical organization of common storage media where the primary memory (RAM) is organized as an array of memory cells, while the secondary memory (hard disk) is organized in sectors or blocks.
  The read and write operations are done by sectors and blocks .
  This storage model guarantees the read/write coherence since the result of a read is the same of the most recent write on the same memory location.
  Cells storage guarantees only before-or-after atomicity, but not all-or-nothing atomicity because once the content of a cell is changed by an action, there is no way to abort the action and restore the original content.
- **Journal Storage Model**: it's thought to manage more complex objects, that are saved as records.
  It's a system composed by a manager that coordinates the accesso to the underlying cell storage.
  In this model we maintain not only the current value but also the entire history of versions of the variables, which is saved in the log.
  The log is provided by an all-or-nothing action that firstly records the action in a log and then installs the changes in the cell storage.
  Journal storage guarantees all-or-nothing atomicity thanks to the log that is done before the action (commit). So if one of these operations is aborted we can return to the previous state.
  All-or-nothing atomicity implies read/write coherence and before-or-after atomicity.

6.1.3 Consensus

Consensus is the process of agreeing to one of several alternatives proposed by a number of agents (in distributed systems the agents are a set of processes that propose a single value).
An important family of protocols to reach consensus is Paxos, which divided the processes in Proposers, Acceptors and Learners and through a set of operations (prepare, promise, accept, learn) they reach consensus.

6.2 Google File System (GFS)

It's designed to guarantee high reliability despite of frequent hardware and software failures.
GFS files can range in size form few GB to hundreads of TB. They are collected in fixed-size segments called **chunks**. Each chunk is of 64MB, is shared on Linux File System and is replicatend on multiple sites (3 by default).
Advantages of large chunks are:

- reduction of metadata maintained by the system;
- reduction of requests to locate the chunks;
- reduction of disk fragmentation.

GFS has a master that controls the chunk servers and maintains metadata such as filenames, access control informations and locations of the replicas.
To recover in case of failures the operation log maintains a historical record of metadata changes. Changes are atomic and become visible only after be recorded on multiple replicas on persistent storage.
To minimize the recovery time, the master periodically checkpoints its state and at recovery time replays only the logs recorder after the last checkpoint.
File access in GFS is handles by the master that grants a lease to a chunk server (primary).The chunk server will be responsible for the updates.
On the other hand, the creation of a file is handles by the master.
The write on a file is done in the following way:

1. Client contacts the master to know which is the primary chunk server and where are the others;
2. Client sends data to all the chunk servers;
3. Chunk servers store data in a LRU (Least Recently Used) buffer and send an ACK to the client;
4. Client sends the write request to the primary;
5. Primary applies the modifications and forward them to the secondaries;
6. Secondaries confirm the operation to the primary;
7. Primary sends an ACK to the client.

6.3 Hadoop Distributed File System (HDFS)

It's a distributed file system written in Java. It's portable but it can't be direclty mounted on an existing OS.
It uses blocks of 64-128MB and replicate them on several nodes (3 by default).
This system follows a master/slave architecture with a master (NameNode) which manages the file system, store metadata, controls access to the files, record changes in a log and checks the DataNodes status.
The slaves are the DataNodes that take care of low level read/write operations.
The write on a file is done in the following way:

1. Client contacts the NameNode to ask for the authorization to write on a file;
2. Client receives the list of the DataNodes for that file;
3. Client contacts directly the primary DataNode;
4. Primary DataNode will contact the secondary and so on;
5. Client starts to send modifications to the primary DataNode;
6. The DataNode applies the modifications and forwards them to the secondary and so on;

7. When each DataNode has applied the modification send an ACK backward on the pipeline, until the client receive all the ACKs.

The read of a file is done in the following way:

1. Client contacts the NameNode to ask for the authorization to read a file;
2. Client receives the list of the DataNodes where each block is stored;
3. Client contacts directly to each DataNode in order to get the blocks;
4. Client will combine the blocks to get the entire file to read.

6.4 Amazon Simple Storage Service (S3)

Data are stored as **objects** into **buckets**. Both objects and buckets are identified by their names.
An user can create a maximum of 100 buckets and he can also set read/write permissions with ACL.
Objects can range fmro 1 to 5 GB and can be loaded (PUT) or downloaded (GET) entirely or by specific blocks. of bytes.

6.4.1 RW failures

The client application must handle the error in read and write, by retrying until the request succeeds. The MD5 checksum is used to check integrity.
In case of write failures, if a PUT request fails, the client has to retry. The request fails if the MD5 received by S3 (which computes it on the file in the bucket) is different from the one that the client application compute on the local file.
In case of read failures, if the MD5, that is attached to the file that S3 sends to the client, is different from the one that the client computes on the received file, the client can retry the read operation.

6.4.2 Data Consistency Model

S3 offers a strong read-after-write consistency both for PUT and DELETE operations, and for ACL, tag and metadata.
Modifies on a single key are atomic: if a thread execute a PUT and an other a GET on the same key, S3 will return either the old or the new data, but never some partially modified data.
A PUT returns a success to the client only when all the replicas have been modified.
If there are two concurrent writer on the same key, the operation with the highest timestamp will win (last-writer-wins). S3 does not support lock mechanism, unless the developer implement it itself.
S3 does not support multi-key operations: so modifies on a key can't affects updates on an other key.
The bucket configuration, instead, follow an eventual consistency model: if a bucket is deleted and someone request the list of buckets, the one deleted can still appear for a short time.

6.5 NoSQL

6.5.1 Databases

Databases are an important component in a cloud application. We can have:

- **Relational DBMS**: guarantees atomicity, consistency, isolation and durability (ACID properties).
  They are reliable but they have problem to work with too many data that require responses in real time.

To mitigate this problem the memcaching mechanism is used to maintain objects in RAM.
They're also complex to scale in speed and size since it's foundamental to keep the consistency between the several copies of the DB.

- **NoSQL Databases**: lots of modern big data do not require a relational model.
  They do not guarantee the ACID properties but the BASE ones (basically available, soft state and eventually consistent).
  They are projected to scale horizontally and to eliminate the single point of failure.

6.5.2 BigTable

BigTable is a distributed storage system developed by Google. It was projected in order to be highly applicable, scalable and performant both for mapreduce operation and for latency-sensitive services.
A BigTable is a sparse, distributed, persistent multidimensional sorted map.
It offers a flexible model where the client controls the layout and format of data and their location.

Rows, Columns and Timestamps

Data are treated as sequences of bytes not interpreted and are indexed using row and column names that can be an arbitrary string.
A row key is ana arbitrary string of up to 64KB.
Each read and write operation on a single row is atomic, regardless from the number of columns involved.
In a table, rows are ordered lexicographically and are dynamically partitioned into **tablets**, that is the fundamental unit for load balancing.
Column keys are grouped into maximum 100 sets called **column families**, where:

- all the data stored in a column family have the same type;
- column family's data can be compressed;
- families should be rarely changed and the data inside can't be added after the family is created;
- access control and memory accounting is done at family level.

A column key is *family:qualifier*.
Each cell can contain multiple versions of the same data, where each version is indeed by a timestamp (64bit integer).
Timestamp can be assigned by BigTable, that uses the real time in microseconds, or by the client, which has to use a unique identifier.
There are two possible mechanism for doing garbage collection: only the last n versions are stored or only the versions created in the last n-days are kept.

Architecture

It consists of one master server and many tablet servers. A BigTable cluster store several tables and each one of them consists of a set of tablets.
The master is responsible for assigning tablets to tablet servers, detecting the addition or expiration of a tablet server, doing load balancing and garbage collection and handling schema changes.
The tablet servers can be dynamically added or removed to accomodate the workload. Each tablet server manages a set of tablets, handles requests for that tablets and splits them if they are too big.

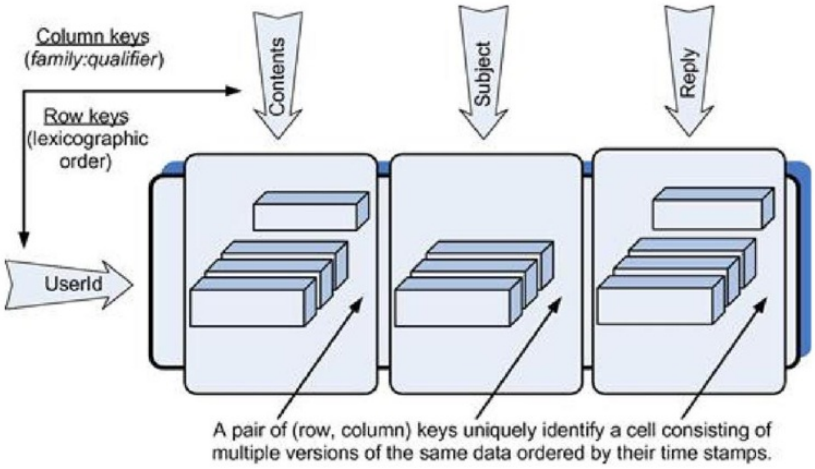


Figure 6.1: BigTable example.

Google SSTable

It's the file format used internally to store Bigtable data. SSTable provides a persistent, ordered immutable key-value map. Each SSTable contains a sequence of blocks, which can be located through a block index and using the binary search.

Chubby

It's a lock service for loosely-coupled distributed system which allows its clients to synchronize their activities and to agree on basic informations about their environment.
Chubby can be used in GFS to elect the master server and to store a small amount of metadata.
Chubby can be used in BigTables to elect the master node, to allow it to discover which tablet server are active and to allow the clients to locate the master and memorize the schema and the ACLs.

6.5.3 Amazon DynamoDB

DynamoDB is the distributed key-value storage of Amazon. It's thought for services that require the access only through primary key, where the relational databases would be inefficient and poor of scaling and availability for those workloads.
Dynamo primary requirement is the high availability and reliability, since the outage would cause significant financial consequences. For this reason Dynamo tradeoffs between availability, consistency, cost-effectiveness and performance.
DynamoDB is eventually consistent: updates spread to all the replicas asynchronously. It eploits optimistic replication techniques: a read/write operation succeeds if a quorum of nodes complete it succesfully. This grant that the system is always ready to receive data (always writable), but introduce the risk of conflicting changes that must be detected and resolved in reading phase. These conflicts can be solved both by the data store (with low logic capacity) and by the client application (more complex solution but also more flexible).

Requirements and Principles

Amazon Dynamo has the following design requirements:

- **Query model**: only simple read and write operations based only on primary keys;
- **No ACID**: Dynamo works with lower consistency to imporove availability;
- **Efficiency**: Dynamo runs on commercial hardware but following the latency and throughput requirements;
- **Security**: there are not specific security requirements since Dynamo is designed to work only in the Amazon network;
- **Scalability**: it has to scale to hundread of storage nodes.

Amazon Dynamo has the following design principles:

- **Incremental scalability**: it has to be able to scale out one storage host at a time;
- **Symmetry**: every node should have the same set of responsabilities as its peers;
- **Decentralization**: favors decentralized peer-to-peer techniques;
- **Heterogeneity**: ability of balancing the workload across the nodes, also considering their diversity.

System Architecture

- **System Interface**: Dynamo exposes only two API primitives:
  - **Get(key)**: locate the replica associate to the key and return the object, or a list of objects if there are version in conflict, with a context;
  - **Put(key, context, object)**: determines on which nodes place the object based on the key and then writes the replica on the disk; the context encodes system metadata (such as the object version)

Partitioning Algorithm

It's used to dynamically partition data over a set of nodes in order to grant horizontally scalability. Dynamo distributes data using consistent hashing: each node get a random value which represents its position inside a "ring". In the same way, each data item receive a position in the ring given by the hash of its key.
The problem is that this method can implies a non uniform and unbalanced distribution over the ring. So the virtual node is introduced: a single physical node does not have a unique position in the ring, but it has multiple virtual nodes (the number depends to its computational capacity compared to the others) with different position. If a node becomes unavailable, its workload is divided across the remaining available nodes. When it becomes newly available (or a new node is added), it will receive a workload proportional with the one of the other nodes.

Replication

To achieve high availability and durability, data is replicated on N hosts. The key is firstly assigned to a coordinator node (the first node found on the ring clockwise). The coordinator will save the key locally and will replicate it on the following clockwise N-1 physical nodes.

Data Versioning

Dynamo manage the data versioning in order to achieve the eventual consistency, which allows asynchronously updates.
Since a put() call can return success before all the replicas are actually updated, a following get() call may return a old data. Some applications (like the Amazon chart) prefers this behaviour than reject an "add to

chart".
When there are concurrent changes, there will be divergent versions that Dynamo will treat as immutable objects. The conflicts will be then reconciled.
In order to track causality and history of the changes and to identify which version depends on which other one, Dynamo uses the Vector Clocks, that are pairs (node, counter).

7. FC1 - Definition and Architecture

7.1 Task 1

The goal is to analyze the following two use cases and determine whether they describe cloud services, using the methodology proposed in NIST SP 500-322.

7.1.1 Use Case 1: Video Conference System

: The Cloud Service Provider (CSP) offers a video conference system as a service.

The CSP offers the following features to the Cloud Service Consumer (CSC):

- CSCs can create their accounts.
- CSCs can use a base service for free (basic users) or can subscribe to an enterprise service that offers advanced features and higher performance (enterprise users)
- The service is accessible from the internet, despite the geographical location of the users, by using a standard browser.
- CSCs can start a video conference with one or more peers, who must be subscribers of the service.
- CSCs can record a conference session and store the audio/video files in a storage area managed by the CSP. There is no limit on the number of audio/video files a user can store
- CSCs can set their video conference preferences that are stored by the CSP.
- Despite the number of CSCs connected to a video conference and the number of simultaneous video conferences, the CSCs always perceive the same quality of service experience.

To provide the above features, the CSP operates as follows:

- The CSP uses a front-end website (accessible through the HTTPS protocol) to let users access the

service and all the service features.

- The CSP stores the users' profiles on its storage system. More storage will be added automatically when the number of users increases.
- The CSP stores the users' files in a dedicated storage system. More storage will be added automatically as the number of users and the amount of storage per user increase.
- When a CSC starts a video conference, the service provider automatically allocates network resources to support the communication and computational resources to run the video conference software. The time required to allocate such resources ranges from a few seconds to 1 minute for enterprise users and from tens of seconds to 2 minutes for basic users.
- In case the number of participants in a video conference increases unpredictably, the CSP will allocate more resources automatically, and this will be transparent to the users, with a setup time of a few seconds (1 – 60)
- The video conference software platform used by the CSP for providing video conferencing is the same for all the users, and each time a video conference is started, the CSP creates a new instance of the software platform. The video conference software platform is a complex system composed of many components that can be deployed on independent virtual servers, and some components can be shared among multiple video conferences running simultaneously. This is possible because the software application uses a multi-tenant model. Moreover, physical and virtual resources instantiated to run the video conferencing software instances are shared among the CSCs
- To better manage the infrastructure, guarantee the agreed level of quality of service, and bill the enterprise users according to the pricing policy, the CSP runs a monitoring platform that allows for measuring the usage of the cloud resources, e.g., data traffic, number of computing resources allocated for a video conference, and storage consumed by a user.

7.1.2 Answers Use Case 1

The first use-case is an example of cloud service since it verifies all the essential characteristics:

- **On-deman self-service**: The computing capability can be provisioned without human interaction with the service provider. The Option A is verified because there is fully automated service provisioning since all the operations are automatically done in max. 2 min in a transparent way for the consumers;
- **Broad Network Access**: The computing capability is available from a wide range of locations using standard protocols. The Option A is verified because the services are available over the Internet (The CSP uses a front-end website (accessible through the HTTPS protocol) to let CSCs access the service and all the service features);
- **Resource Pooling**: The computing infrastructure is shared among more than one CSC. Two or more CSCs can share the cloud service resources using a multi-tenant model; in fact the video conference software platform used by the CSP for providing video conferencing is the same for all the users, and each time a video conference is started, the CSP creates a new instance of the software platform. This is possible because the software application uses a multi-tenant model. Moreover, physical and virtual resources instantiated to run the video conferencing software instances are shared among the CSCs;
- **Rapid Elasticity**: The computing capabilities can be "rapidly" provisioned and released to scale. It follows the Option A since resource allocation modification is automated and near-real-time (In case the number of participants in a video conference increases unpredictably, the CSP will allocate more

resources automatically, and this will be transparent to the users, with a setup time of a few seconds (1 – 60));

- **Measured Service**: Cloud service characteristics including resource usage are measured with enough detail to support the requirements of the CSC. To better manage the infrastructure, guarantee the agreed level of quality of service, and bill the enterprise users according to the pricing policy, the CSP runs a monitoring platform that allows for measuring the usage of the cloud resources, e.g., data traffic, number of computing resources allocated for a video conference, and storage consumed by a user.

The cloud computing service model used is SaaS (Software as a Service) since the service that is provisioned is a software application, described as computer programs designed to permit the user to perform a group of coordinated functions, tasks, or activities.
This is an example of public cloud because unrelated CSCs use the shared cloud service and the underlying resources.

7.1.3 Use Case 2: Scientific Computing Platform

The CSP offers to the CSC a scientific computing platform as a service. The platform has the following features:

- The service is accessible from the Internet, despite the geographical location of the CSC, by using a standard browser and standard Internet protocols.
- CSCs can create their accounts.
- CSCs have a web interface to select the resources they want to use. A resource is a Virtual server with scientific software installed. There is a catalog with a broad choice of different combinations of Virtual servers and scientific software. Each combination has a different price per hour of usage.
- When the selection of the resource is completed, the CSC can proceed to the checkout. That means the CSP is informed about the resources to instantiate.
- When a set of resources is instantiated for a CSC, he receives a notification by email with the information needed to access the virtual server via an SSH connection. The CSP guarantees that a CSC request will be served within 36 hours of the checkout time.
- The CSC will pay for the instantiated resources even if not used. If more or different resources are needed, the CSC should go through the selection process described before

To provide the above features, the CSP operates as follows:

- The CSP uses a front-end website (accessible through the HTTPS protocol) to let CSCs access the service and all the service features.
- The CSP stores the CSCs' profiles on its storage system. More storage will be added automatically when the number of CSC' profiles increases.
- The CSP stores the CSC files in a storage volume mounted on the virtual machine instantiated for the CSC. More storage will be added automatically when the number of files increases.
- The CSP operates a back-end office active 365/24/7 where human operators process the CSCs' requests to instantiate virtual servers. The human operators verify that the CSP has sufficient physical servers and storage to accommodate the CSC request, then proceed with automatic instantiation and notification. If no physical resources are available, the request is placed in a queue. The CSP is confident that, within 36 hours, the CSC request can be served.
- The CSCs share the physical machines and storage operated by the CSP. Physical resources are assigned

and reassigned based on CSC demand.

- To bill the CSC according to the pricing policy, the CSP runs a monitoring platform that allows the measurement of the usage of physical and virtual resources.

7.1.4 Answers Use Case 2

The second use-case is not an example of cloud service since it not verifies all the essential characteristics:

- **NO On-deman self-service**: The computing capability are provisioned with human interaction of the service provider. The CSP operates a back-end office active 365/24/7 where human operators process the CSCs' requests to instantiate virtual servers. The human operators verify that the CSP has sufficient physical servers and storage to accommodate the CSC request, then proceed with automatic instantiation and notification. If no physical resources are available, the request is placed in a queue. The CSP is confident that, within 36 hours, the CSC request can be served;
- **Broad Network Access**: The computing capability is available from a wide range of locations using standard protocols. The Option A is verified because the services are available over the Internet (The CSP uses a front-end website (accessible through the HTTPS protocol) to let CSCs access the service and all the service features);
- **Resource Pooling**: The computing infrastructure is shared among more than one CSC. Two or more CSCs can share the cloud service resources using a multi-tenant model; in fact the CSCs share the physical machines and storage operated by the CSP. Physical resources are assigned and reassigned based on CSC demand;
- **NO Rapid Elasticity**: The computing capabilities are not "rapidly" provisioned and released to scale. When a set of resources is instantiated for a CSC, he receives a notification by email with the information needed to access the virtual server via an SSH connection. The CSP guarantees that a CSC request will be served within 36 hours of the checkout time;
- **NO Measured Service**: Cloud service characteristics including resource usage are measured with enough detail to support the requirements of the CSC but the CSC will pay for the instantiated resources even if not used.

The service model used is IaaS (Infrastructure as a Service) since the service that is provisioned is infrastructure. This is an example of public cloud because unrelated CSCs use the shared cloud service and the underlying resources.

7.2 Task 2

7.2.1 Deconstructing the prerequisities

The authors argue that scalability is a "prerequisite" for elasticity. Explain why a system can be scalable without being elastic, and specifically identify the "temporal aspects" that elasticity introduces which scalability lacks.:

A system is scalable if it has the ability of sustaining increasing workloads with adequate performances, adding hardware resources. Scalability is a prerequisite for elasticity, but it does not consider temporal aspects of how fast, how often, and at what granularity scaling actions can be performed. Scalability is the ability of the system to sustain increasing workloads by making use of additional resources, and therefore, in contrast to

elasticity, it is not directly related to how well the actual resource demands are matched by the provisioned resources at any point in time. So a system can be scalable but not elastic.

7.2.2 The Matching Criterion

According to the paper's refined definition, elasticity is measured by how closely available resources match current demand. Analyze the "Matching Function" (m(w)=r). Why must this function be derived separately for both upward and downward scaling, rather than assuming a symmetrical relationship?.:
The matching function m(w)=r represents the minimum amount of resources r required to satisfy performance requirements for a given workload intensity w. It essentially describes the ideal resource allocation for each workload level. The paper states that this function must be derived separately for upward scaling (scale-out/up) and downward scaling (scale-in/down) because the relationship between workload and resource allocation is not symmetrical in real systems. The system reacts differently depending on whether the workload is increasing or decreasing:

- **Upward scaling**: When workload increases, the system may add resources only after performance thresholds are exceeded;
- **Downward scaling**: When workload decreases, the system usually waits longer before removing resources to avoid oscillations (rapid scaling up and down).

7.2.3 Metrics of Speed vs. Precision

Distinguish between "Speed" and "Precision" as core aspects of elasticity. How would a system that scales "rapidly" but with low "precision" impact both the cloud provider and the end-user?:
Speed: The speed of scaling up is defined as the time it takes to switch from an underprovisioned state to an optimal or overprovisioned state. The speed of scaling down is defined as the time it takes to switch from an overprovisioned state to an optimal or underprovisioned state. The speed of scaling up/down does not correspond directly to the technical resource provisioning/deprovisioning time.
Precision: The precision of scaling is defined as the absolute deviation of the current amount of allocated resources from the actual resource demand. If a system scales "rapidly" (high speed) but with "low precision," it means that the system reacts to workload changes almost instantly, but it consistently fails to calculate and allocate the correct amount of resources. When a low precision system undershoots the actual demand, it fails to provide the minimal amount of resources (r) required to satisfy the system's performance requirements for the current workload intensity (w). The system enters a state of underprovisioning. Because there are fewer resources available than what the workload requires, the system cannot process all incoming requests efficiently. The end-user will experience a severely degraded quality of service, such as slow application response times, timeouts, or complete service unavailability. Conversely, when a low precision system overshoots the actual demand, it allocates far more resources than the workload actually needs. The system enters a state of overprovisioning. The cloud provider ends up powering, dedicating, and maintaining physical infrastructure (such as discrete units of CPU cores or Virtual Machines) that sits completely idle. This results in massive hardware waste and unnecessary financial costs, while also preventing them from leasing those idle resources to other paying customers.

7.2.4 Elasticity vs. Efficiency

Using Figure 3 as a reference, explain why a more efficient system might appear to have "better" elasticity even if its adaptation mechanisms are technically inferior. How do the authors suggest a benchmark should account for this to ensure a "fair comparison"?:

Efficiency expresses the amount of resources consumed for processing a given amount of work. A more

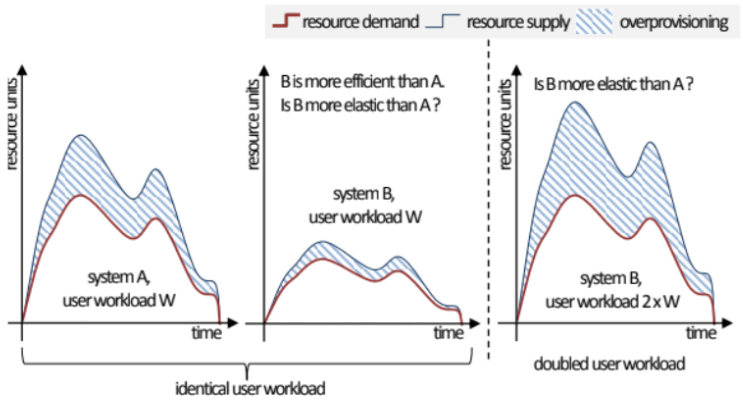


Figure 3: Elasticity vs. Efficiency

efficient system consumes fewer resources for the same work. If we have two systems tested with the same workload, the more efficient one would seems as the more elastic, but only because its adaptation mechanism is not stressed to the same extent as the other system.

To enusre fairness, a benchmark must induce identical demand curves rather than just identical workloads. This requires the benchmark to adjust the workload for the more efficient system (for example doubling the workload for it) so that both the system adaptation mechanisms are stressed at similar intesities.

7.2.5 The Impact of Discrete Scaling Units

The authors note that resources are typically provisioned in discrete units (e.g., VMs, CPU cores). How do these "Resource Scaling Units" prevent a system from ever achieving "perfect" or "optimal" elasticity, even if the adaptation process is instantaneous?:

In real-world scenarios, a system's workload intensity such as incoming user requests or data traffic fluctuates in a smooth, continuous manner. However, cloud systems cannot provision resources in perfectly fluid fractions. As the authors note, resources of a given type can only be provisioned in rigid, discrete units, such as individual CPU cores, Virtual Machines (VMs), or physical nodes. The authors describe optimal elasticity as a hypothetical, perfect scenario where a system has no upper bounds on available resources, and it provisions and de-provisions these resources immediately as they are needed, matching the actual demand exactly at any given point in time. Even if we assume this adaptation process is instantaneous (zero delay in booting up a server or releasing it), a fundamental mathematical mismatch remains. Because demand is continuous but supply must be delivered in blocks (the Resource Scaling Units), the supply curve will always look like a "staircase," while the demand curve looks like a smooth wave. Therefore, even with a technically instantaneous and flawless adaptation mechanism, absolute precision is impossible. There will always be a

residual deviation between the resources allocated and the exact resources needed.

7.2.6 Benchmarking Real-Life Scenarios

The paper suggests that demand curves for benchmarking must include "bursts," "trends," and "seasonal patterns". Select one of these patterns and explain which specific elasticity metric— $\bar{A}$ (average time to scale up) or Pd (average precision of scaling down)—would be most critical for a system to maintain performance during that specific workload behavior.:

When evaluating a system's ability to handle sudden "bursts" of workload intensity, the most critical metric for maintaining performance is A (the average time to scale up).

A burst represents a sudden, sharp increase in workload intensity. This rapid surge immediately pushes the system into an underprovisioned state, meaning the actual demand exceeds the currently provisioned resources.

The paper defines A as the average time it takes for a system to switch from an underprovisioned state to an optimal or overprovisioned state. In practical terms, this represents the "average speed of scaling up".

To maintain performance during a burst, the system must provision new resources as fast as possible (minimizing A) to handle the influx of requests.

Therefore, a fast A is essential to ensure the system doesn't buckle under a sudden burst, while P_d is more relevant to the economic efficiency of the system once that burst subsides.

7.3 Sum-up

7.4 Chapter Recap and Exam Preparation

7.4.1 Cloud Service Evaluation (NIST SP 500-322 / 800-145)

To classify a computing service as a cloud service, it must fulfill all five essential characteristics defined by NIST:

- **On-demand self-service:** Consumers provision computing capabilities automatically without human interaction from the service provider.
  - *Option A:* Fully automated process from user interface to internal infrastructure (e.g., Video Conference System allocating resources instantly).
  - *Option B:* User interface is automated, but internal backend provisioning requires human operator intervention (e.g., Scientific Computing Platform where a human office processes requests, causing a 36-hour queue delay; this fails the official on-demand standard).
- **Broad network access:** Capabilities are available over the network and accessed through standard mechanisms (e.g., web browsers, standard HTTPS/SSH protocols) by heterogeneous client platforms, regardless of geographical location.
- **Resource pooling:** Provider resources are pooled to serve multiple consumers using a multi-tenant model, with resources dynamically assigned according to demand. Multi-tenancy can occur at the application layer (shared software platform) or infrastructure layer (shared physical servers/storage).
- **Rapid elasticity:** Capabilities can be elastically provisioned and released to scale rapidly outward and inward commensurate with demand.

  - *Option A:* Near-real-time automated resource modification to handle unpredictable workload spikes.
  - Systems requiring a manual resource re-selection process or introducing massive time windows (e.g., 36 hours) lack rapid elasticity.
- **Measured service:** Cloud systems automatically control, optimize, and meter resource utilization (monitoring, reporting, billing). True cloud services bill users on a flexible pay-per-use/charge-per-use basis. If a system charges for resources when they sit completely idle, it violates this economic principle.

7.4.2 Cloud Service and Deployment Models

- **Software as a Service (SaaS):** Complete software applications provided over the network, intended for end-users (e.g., Video Conference System).
- **Platform as a Service (PaaS):** Provides development environments, tools, or deployment stacks for application developers.
- **Infrastructure as a Service (IaaS):** Provides fundamental computing resources (virtual servers, storage, networks) where consumers can run arbitrary operating systems and software (e.g., Scientific Computing Platform).
- **Public Cloud:** The cloud infrastructure is provisioned for open use by the general public, owned and operated by the provider, and shared among unrelated consumers.

7.4.3 Elasticity Fundamentals, Metrics, and Benchmarking

- **Scalability vs. Elasticity:** Scalability is the baseline ability of a system to sustain increasing workloads by adding hardware resources. A system can be scalable without being elastic. Elasticity introduces **temporal aspects** (speed and granularity) focusing on real-time, automatic provisioning/deprovisioning to match demand precisely.
- **Matching Function ($m(w) = r$):** Returns the minimal resources (r) needed for a specific workload (w) to meet performance thresholds. It must be derived separately for upward and downward scaling because real systems behave asymmetrically (scaling up quickly to safeguard performance, scaling down with a delay to avoid rapid, inefficient oscillations).
- **Speed vs. Precision:**
  - *Speed:* The time required to switch between under/overprovisioned states and the optimal state.
  - *Precision:* The absolute deviation between allocated resources and actual real-time demand.
  - *Impact of rapid but low-precision scaling:* Leads to severe mismatches. If it undershoots (**underprovisioning**), the end-user experiences degraded service quality, high latency, or time-outs. If it overshoots (**overprovisioning**), the cloud provider incurs massive hardware waste and financial costs due to idle resources.
- **Resource Scaling Units:** Real user demand fluctuates in a smooth, continuous wave, but cloud resources are provisioned in discrete, rigid units (e.g., integer numbers of VMs or CPU cores). This discrete nature creates a permanent residual deviation, preventing the system from ever achieving theoretically "perfect" or "optimal" elasticity.
- **Elasticity vs. Efficiency:** Efficiency measures the raw amount of resources consumed to process a given workload. A highly efficient system consumes fewer resources and might falsely appear to have "better" elasticity because its scaling mechanisms are less stressed. To guarantee a fair comparison, a

benchmark must induce identical resource demand curves by calibrating the workload intensity applied to each system.

- **Workload Behavior (Sudden Bursts):** A burst represents a sharp, immediate spike in user requests that instantly causes underprovisioning. To maintain performance and prevent crashes during a burst, the most critical metric is $\overline{A}$ (average time to scale up). Metrics like P_d (average precision of scaling down) only become relevant later for economic efficiency once the burst subsides.

8. FC2 - Autonomic Computing

8.1 Task 1 - Foundations and Ispirations

8.1.1 The Biological Metaphor

Explain how the human Autonomic Nervous System (ANS) serves as the primary inspiration for autonomic computing. Contrast the role of the "conscious brain" in humans with the role of a "human administrator" in a computing system.:

Autonomic Comuting takes inspiration from the human Autonomic Nervous System (ANS), which regulates vital functions (like body temperature and oxygen levels) subconsciously to ensure survival. In this metaphor:

- **ANS = Autonomic Manager**: Controls subconscious, low-level activities without wilful human control.
- **Conscious Brain = Human Administrator**: Specifies high-level objectives and only intervenes for major failures or to change system goals.

8.1.2 Homeostasis vs. Vital Parameters

Define homeostasis in the context of both biological and computing systems. What are the "vital parameters" a computing system must maintain within a "survivability zone"?:

Homeostasis in the context of biological systems is a system's ability to maintain internal equilibrium despite changes in its external environment and its internal state. The ANS uses external and internal sensory information to regulate the activity of internal organs so as to maintain a set of vital parameters (body temperature, oxygen levels or glucose concentration) within a 'survivability' zone. In a computing system homeostasis refers to the process of maintaining "vital parameters" within a "survivability zone" without human intervention. These parameters could be functional services and Quality of Service (QoS) properties.

8.1.3 The Role of Motility

According to Section 3.1, how did the biological emergence of motility (movement) necessitate the development of a nervous system? How does this relate to the "change management" required in dynamic software environments?:
While static organisms like plants and fungi were rooted in their environments and did not have a stringent need for a nervous system, the emergence of animal motility brought perpetual environmental shifts. An organism's survival depended on its ability to adapt to new conditions and avoid fatal environments. The nervous system developed as a crucial enabler for this movement, coordinating the actions of various body parts and acting as a "change manager" to help the animal navigate intelligently.
Similarly, software programs were like static organisms; they were isolated entities confined to static environments, carrying out strictly defined calculations. Today, programs are becoming increasingly "agile and mobile," operating collaboratively within highly unpredictable and dynamic environments. Just as the biological nervous system evolved to manage the changes brought on by physical motility, modern software engineering draws inspiration from this biological model. It demonstrates how interconnecting basic system elements into "rapid communication circuits" can allow complex software systems to efficiently and coherently adapt to their ever-changing execution environments.

8.1.4 Scientific Cross-Pollination

Autonomic computing is a consolidation of several fields. Identify three non-computing scientific domains (e.g., from Section 3.1) that contribute models to this field and provide one example of a concept from each.:
Autonomic computing consolidates theories from various domains:

- **Economics**: Virtual currencies for negotiation among processes;
- **Biology**: The ANS metaphor and homeostasis;
- **Control Theory**: Use of feedback loops for regulation.

8.1.5 Microeconomics in Resource Allocation

How are concepts from microeconomics, such as "virtual currencies" and "supply and demand patterns," applied to solve resource allocation problems among competing autonomic processes?:
The concepts of virtual currencies and supply demand patterns are applied through the following mechanisms:

- **Virtual Currencies**: In posted-price negotiation, the provider sets a price in real or artificial currency for its service, and the requester must take it or leave it.
- **Auctions and Brokers**: Autonomic systems will be inhabited by middle agents that serve as intermediaries of various types, including... brokers, auctioneers. These third-party arbiters "can run an auction or otherwise assist these more complex negotiations" over limited resources.
- **Utility Functions**: To manage supply and demand conflicts, Utility functions rely on the definition of a quantitative level of desirability of a given system state. This ensures that when insufficient resources are available, the most desirable combination of available resources can be found.
- **Tendering (Market Dynamics)**: To balance dynamic supply and demand, a process called tendering is used to select the node. During this process, the sensor node with the Quality of Context closest to that requested wins the tender, and frequent retendering allows the requester nodes to autonomously adapt to the best source available.

8.1.6 Nash Equilibrium vs. Pareto Optimality

In a multi-agent autonomic system, explain why a Nash Equilibrium (local optimum) might not be Pareto optimal (global optimum). What is required to move a system toward the Pareto boundary?:
In a Nash Equilibrium, every application in the autonomic system makes the best possible decision for itself, assuming no other application changes its behavior. Because applications act selfishly to maximize their own resources, they often settle into a state that harms the overall system's performance. No single app can fix the issue alone, leaving the system stuck in a suboptimal state.
The Pareto Optimality is The Global Optimum: a system reaches the Pareto boundary when it is operating at peak collective efficiency. In this global optimum, it is impossible to give one application more resources without taking them away from another.
Moving from a selfish Nash Equilibrium to a globally efficient Pareto state fundamentally requires cooperation among the agents. Since perfect knowledge of every other agent's strategy is usually impossible in complex systems, the text suggests two workarounds. First, the Stackelberg Leadership model allows one "leader" agent to use its private knowledge to optimize its own responses, which inherently improves the performance of all other agents. Second, Conjectural Equilibrium allows agents to rely on "beliefs" about the system, learned through repeated interactions with their environment, instead of requiring perfect knowledge.

8.1.7 Stackelberg vs. Conjectural Equilibrium

Distinguish between Stackelberg leadership and Conjectural equilibrium. In what scenarios is Conjectural equilibrium preferred due to "imperfect knowledge" of a competitor's strategy?:
Stackelberg Leadership describes a scenario where one specific player acts as a "leader" because they possess private knowledge of all their competitors. By having this broader view, the leader can optimize its responses in a way that coordinates the system. Interestingly, this approach often improves the performance of every player involved, even if the "followers" continue to act myopically without understanding the bigger picture.
Conjectural Equilibrium serves as a decentralized alternative for systems where no leader exists and perfect knowledge of others is impossible. In this model, agents replace concrete knowledge with "beliefs" or conjectures about how their competitors will act. These beliefs are not static; they are developed and refined over time through repeated interactions and feedback from the environment.
Conjectural equilibrium is preferred when "imperfect knowledge" prevents agents from knowing the private strategies or internal states of their competitors. This is particularly useful in large-scale, open autonomic systems where it is computationally expensive or security-prohibited for applications to share their internal logic. In these cases, agents cannot calculate a perfect response, so they must instead "learn" to coexist by observing the results of their actions and adjusting their beliefs about the rest of the system accordingly.

8.1.8 Complex Adaptive Systems (CAS)

Define a Complex Adaptive System in the context of Section 3.5. Why is it critical for researchers to distinguish between a system being "extremely complicated" and one having "nonlinear composition"?:
A CAS is a system that is both complex and adaptive, modifying itself in response to environmental changes and learning from the experience. An extremely complicated system is one composed of many parts, while a non linear composition system is one which is not simply a sum of its parts.

8.1.9 Emergence and Self-Organitation

Explain the relationship between self-organization and emergence in CAS. How can simple local interactions between autonomic agents lead to complex global behaviors?:
In a Complex Adaptive System (CAS), self-organization and emergence operate as the "how" and the "what" of decentralised order. Self-organization is the process where a system's internal components interact to create structure or behavior without any external direction or central controller. Emergence is the resulting phenomenon—the complex, high-level patterns or properties that appear at the global level which are not present in, and cannot be predicted by, the individual parts in isolation. Essentially, self-organization is the engine of interaction, and emergence is the surprising collective result it produces.

8.1.10 Ashby's Ultra-Stable System

Describe W. Ross Ashby's double-feedback control system. How does the second loop (meta-management) differ from the first loop (reactive) in terms of the time scale and its impact on "essential variables"?:
Ashby's model is designed to keep "essential variables"—the parameters necessary for survival or stability—within a specific "viability zone.":

1. **The First Loop: Reactive Feedback**:
   - **Function**: This loop acts as a direct response mechanism. It uses a "complicated sensor and actuator system" to oppose external disturbances.
   - **Time Scale**: It operates on the short term. It provides immediate, default reactions to common environmental fluctuations.
   - **Impact on Essential Variables**: It monitors the variables and triggers a reaction to "bring them back within the physiological zone" as soon as they are pushed toward a limit.
2. **The Second Loop: Meta-Management**:
   - **Function**: This is a higher-level loop that monitors the success of the first loop. If the default reactions fail to stabilize the system (usually due to dramatic environmental changes), this loop triggers an adaptation in the first loop's behavior.
   - **Time Scale**: It intervenes over a larger (longer) time scale. It does not react to every small disturbance but rather evaluates the trend of the system over time.

8.2 Task 2 - Autonomic Architectures

8.2.1 The MAPE-K Loop Anatomy

Diagram and explain the four functional parts of the MAPE-K loop: Monitor, Analyze, Plan, and Execute. Why is the Knowledge (K) component considered the "heart" of this architecture?:
Autonomic Manager is composed by 4 part:

1. **Monitor**: This component collects, aggregates and filters details from the managed element and its external environment.
2. **Analyze**: This part provides the mechanisms to observe and understand situations. It correlates the data from the Monitor to determine if the system is meeting its goals or if changes are required to maintain peak performance
3. **Plan**: Based on the analysis, this component constructs the action plans needed to achieve objectives or recover from failures.

4. **Execution**: This component is responsible of the real execution on the managed element. Based on the analysis, this component constructs the action plans needed to achieve objectives or recover from failures.

The **Knowledge (K)** component acts as the central repository, that connect all component together and allows them to work as a single component.
It senses when a variable becomes critical (reaches the absolute viability limit). Instead of directly fixing the variable, it reconfigures the reactive behavior of the first loop so that the system can find a new way to stay within limits.

8.2.2 Touchpoints: Sensors and Effectors

What is the role of touchpoints in an autonomic architecture? How do they allow a manager to apprehend a "managed artefact" that is otherwise too complex to be completely known?:
Touchpoints are the interfaces between the autonomic manager and the managed artefacts. They allow the manager to apprehend a resource's state (via sensors) and influence it (via effectors) without needing to know the resource's internal complexities.

8.2.3 Innate vs. Acquired Knowledge

Distinguish between innate knowledge (engraved in the manager) and knowledge by acquaintance (captured via touchpoints). Provide an example of each in a cloud computing context.:

- **Innate Knowledge (A Priori)**: Knowledge "engraved" into the system by designers and architects before the system ever runs.

  Example
  "If the CPU exceeds 80% of usage, then add a new VM" , the system knows a priori this information.

- **Knowledge by Acquaintance (Acquired)**: Information obtained during runtime through direct interaction with the environment and the managed system.

  Example
  The number of people that are online right now on our system "There are 4200 users online". The system learns this info by looking at the traffic.

8.2.4 Rule-Based (Reflex) Reasoning

Explain the Event-Condition-Action (ECA) rule mechanism. What are the primary advantages and limitations of using reflex-based reasoning for system adaptation?:
Reflex-oriented managers are typically stateless managers that react to external events in a reflex, non-deliberative manner. In general, such managers are essentially made of event–condition–action (ECA) rules that directly produce adaptation plans (actions) from specific event and condition combinations. Writing adaptation policies is fairly straightforward but can become a tedious task for larger complex systems. Its simplicity and rapidity remains its biggest strength. However, an issue with ECA rules is the problem of

conflicts: an event might satisfy the conditions of two different ECA rules, yet each rule may dictate an action that conflicts with the other. Worse, these conflicts cannot always be detected at the time of writing the policies; some are only detected at runtime. This means that a human may remain in the loop to solve policy conflicts when they arise.

8.2.5 Model-Driven Autonomicity

What is "model-driven autonomicity," and what are the three types of models typically maintained by a manager (structure, environment, non-functional)? How does the separation of concerns apply here?:
Model-driven autonomicity is an approach where an autonomic system uses synchronized models at runtime (model@runtime) to drive its self-management processes. This strategy separates data from logic, allowing the MAPE loop to focus on the "know-how" (decision-making) instead of data collection and representation. The three types of models typically maintained by a manager are:

- **Structure (Architecture) Model**: Represents the software components and their interconnections.
- **Environment Model**: Represents the external context and computing resources where the system operates.
- **Non-Functional Model**: Focuses on specific quality attributes such as security and performance.

Separation of concerns is applied by keeping these models distinct but linked. This allows each model to evolve independently (e.g., updating security rules without changing the architecture), making the manager's code leaner and easier to modify.

8.2.6 Goal-Based vs. Utility-Based Systems

Contrast a goal-based system (which seeks a target state) with a utility-based system (which ranks states). In what scenario would a utility function be necessary to resolve conflicting goals?:
A Goal-Based System classifies all systems strictly as desirable or undesirable. If an ideal state is impossible to reach, the system does not know which among the undesirable states is least bad. Indeed an Utility-based System relies on a quantitative level of desirability of a given system state. They calculate the relative Utility of each combination allowing the system to rank and compare different states. An Utility function is needed to resolve conflicting goals when insufficient resources are available to satisfy all system demands simultaneously. It mathematically allows the autonomic manager to find the best combination of available resources.

8.2.7 Search-Based Reasoning

In Chapter 4, search-based reasoning is described as finding a "path" from a current state to a target state. What are the three components required to define such a search problem?:
To define a search problem within the context of autonomic computing, the text identifies the following three essential components:

1. **Initial and Target States**: The problem begins with a description of the initial state, which represents the current situation of the managed artefacts as captured through system touchpoints. This is paired with a description of the acceptable target states, or goals, which define what the system is trying to achieve.
2. **Set of Actions**: The system must have a defined set of actions that can be performed on the managed artefacts. These actions are executed via the effectors provided by the system and allow the managed resources to be manipulated to effect change.

3. **Transition Model**: A transition model is required to define exactly which actions must be realized to move the system from one state to another. This model essentially maps out the logic of how individual actions result in state changes, allowing the algorithm to navigate the problem space.

8.2.8 Architectural Models and Runtime Reflectivity

How does an Architectural Model reflect the structure and requirements of a managed system? Why is it important that this model is kept "causally connected" to the running software?:
An architectural model represents the managed system as a logical graph where nodes denote components (computational tasks) and arcs denote connectors (communication paths). It reflects system requirements by defining a set of constraints and properties rather than a fixed configuration; for instance, it might mandate a minimum number of active server components to satisfy throughput goals. Additionally, the model mirrors the operating environment by incorporating sensor data such as network latency or user input, allowing the manager to reason about the system's context.
The "causal connection" ensures that the model and the running software remain synchronized, which is fundamental for integrity verification. Because the autonomic manager applies and verifies adaptation plans on the model first, it can guarantee that a change will not violate system constraints before it is executed on the live resources. This connection is also the primary defense against the stale state paradox: without tight synchronization, an autonomic manager might execute a repair plan based on an outdated "belief" about the system state (e.g., adding a server to fix an overload that has already passed), potentially destabilizing the environment.

8.2.9 Probabilistic Reasoning and Uncertainty

Explain the use of Probability of Correctness (PoC) when dealing with multiple Context Providers (CPs). How does combining data from different sensors (like a camera and a pressure pad) reduce uncertainty?:
In dynamic environments, data from sensors is often uncertain or conflicting. Using Probability of Correctness (PoC) allows the system to weigh inputs from different "Context Providers" to reach a reliable conclusion despite "noisy" or incomplete information.

8.2.10 Autonomic Systems That Learn

Describe the most ambitious type of autonomic system—one that includes learning capabilities. How can reinforcement learning be used to update a manager's knowledge or even its utility functions?:
Learning systems use techniques like reinforcement learning (inspired by the biological dopamine reward signal) to improve their performance over time. This allows a generic manager to start with basic "innate" rules and develop sophisticated, specialized behaviors for its specific environment.

8.2.11 Self-management In Practice

Consider the following scenarios and explain which of the self-management capabilities are implemented.:

- **Scenario 1:**
  Self-Optimization: the Autonomic Manager constantly analyzes performance metrics (CPU utilization, response time) to balance conflicting objectives: guaranteeing performance thresholds (SLOs) while simultaneously ensuring the cost of virtual resources remains minimal.

- **Scenario 2:**
  Self-Healing: the Autonomic Manager achieves this by continuously monitoring the "health state" of the containerized application instances. If it detects that a failure has caused the number of healthy instances to drop below the required threshold of seven, it recovers the system by starting a replacement instance.

8.3 Sum-up

8.3.1 Foundations and Inspirations

- **The Biological Metaphor**: Autonomic Computing is inspired by the human Autonomic Nervous System (ANS), which maintains vital functions subconsciously.
  - **ANS = Autonomic Manager**: Controls low-level, subconscious activities automatically.
  - **Conscious Brain = Human Administrator**: Sets high-level goals and policies, intervening only for major updates or changes in strategy.
- **Homeostasis**: The ability of a system to maintain internal equilibrium despite external changes.
  - **Vital Parameters**: Properties like functional services and Quality of Service (QoS) metrics.
  - **Survivability Zone**: The boundaries within which these parameters must stay without human help.
- **Motility and Change Management**:
  - In biology, movement (motility) brought environmental changes, making the nervous system necessary to navigate safely.
  - In software, programs moved from isolated, static calculations to mobile, dynamic, and distributed environments. Modern systems use rapid communication circuits to adapt quickly as a “change manager”.
- **Scientific Cross-Pollination**: Autonomic Computing combines models from multiple fields:
  - **Economics**: Uses virtual currencies and market models for resource negotiations.
  - **Biology**: Provides the ANS metaphor, immune models, and homeostasis principles.
  - **Control Theory**: Applies feedback loops to regulate system variables.
- **Microeconomics in Resource Allocation**:
  - **Virtual Currencies**: Used in posted-price negotiations where a provider sets a take-it-or-leave-it price.
  - **Auctions and Brokers**: Middle agents act as third-party arbiters to coordinate complex negotiations.
  - **Utility Functions**: Mathematical definitions of the desirability of a given system state. They rank combinations to find the best state when resources are limited.
  - **Tendering**: A market dynamic where requester nodes broadcast demands, and the sensor node with the closest Quality of Context (QoC) wins the contract. Frequent retendering allows autonomous adaptation.
- **Game Theory and Equilibrium Models**:
  - **Nash Equilibrium vs. Pareto Optimality**: In a Nash Equilibrium, agents act selfishly and maximize their own utility, assuming others do not change behavior. This creates a local optimum that harms collective efficiency. Pareto Optimality is the global optimum (Pareto boundary) where

it is impossible to improve one agent without hurting another. Moving to the Pareto boundary requires cooperation.
  - **Stackelberg Leadership**: One "leader" agent uses private knowledge of competitors to optimize its responses, implicitly coordinating and helping all "follower" agents.
  - **Conjectural Equilibrium**: A decentralized alternative where agents lack perfect knowledge and replace it with "beliefs" (conjectures) about competitors. These beliefs are refined through runtime environmental interactions. It is preferred when computational costs or security rules prevent sharing internal logic.
- **Complex Adaptive Systems (CAS)**:
  - **Definition**: Systems composed of many interacting parts that modify themselves and learn from environmental experiences.
  - **Complicated vs. Nonlinear**: An extremely complicated system simply has many parts. A nonlinear system is not just the sum of its parts; interactions create complex, non-additive behaviors.
  - **Self-Organization**: The internal process where components interact to build structure without central or external control (the engine).
  - **Emergence**: The resulting global phenomenon, pattern, or property that arises collectively and cannot be predicted by looking at individual parts alone (the outcome).
- **Ashby's Ultra-Stable System**: A double-feedback control mechanism to keep essential variables viable:
  - **Reactive Loop (First Loop)**: Short-term, immediate action. Uses sensors and actuators to counter common external disturbances and bring variables back to the physiological zone.
  - **Meta-Management Loop (Second Loop)**: Longer time scale. It monitors the success of the first loop. If the first loop fails to stabilize the system under major environmental shifts, it alters the first loop's configuration or rules.

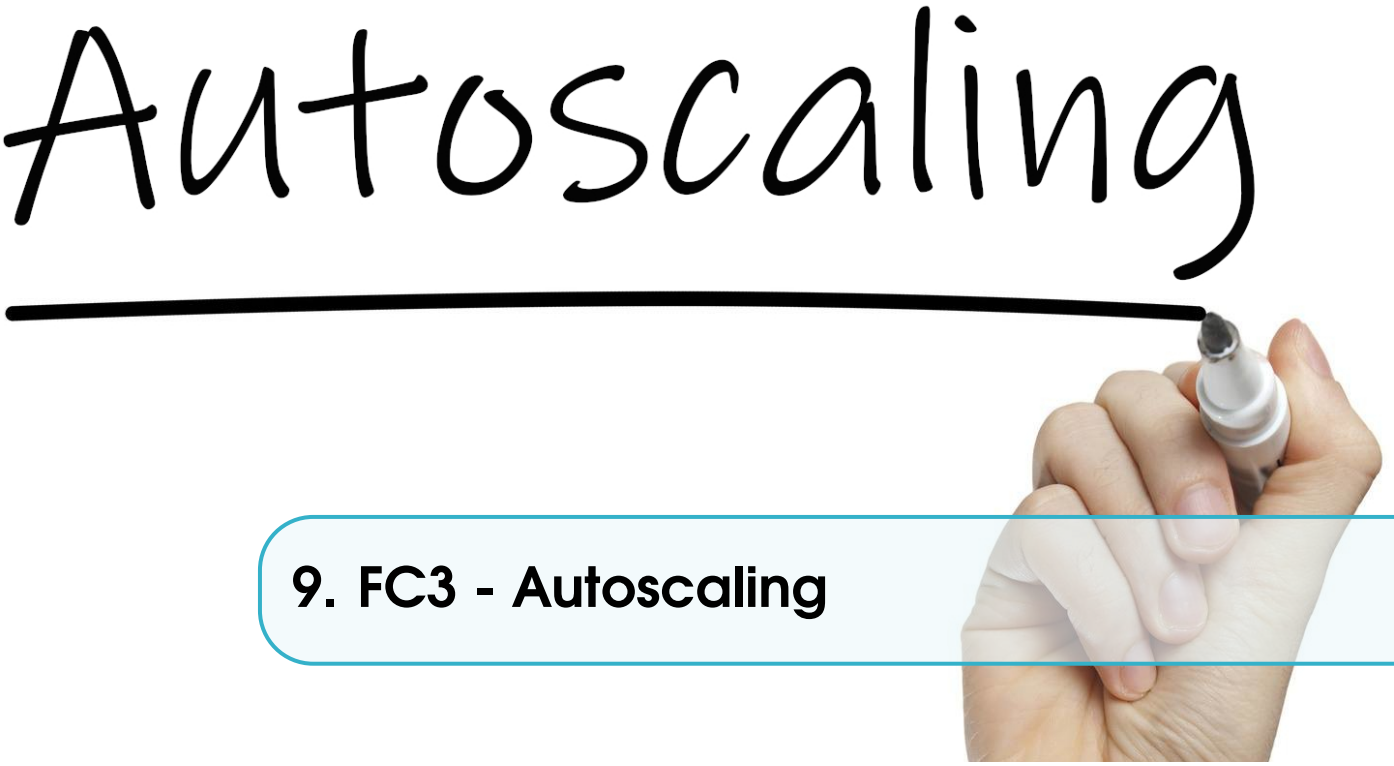
8.3.2 Autonomic Architectures

- **The MAPE-K Loop Anatomy**: The logical reference architecture for an autonomic manager:
  1. **Monitor**: Gathers, filters, and aggregates details from managed elements and the context.
  2. **Analyze**: Understands situations, correlates data, and flags when constraints or goals are breached.
  3. **Plan**: Constructs action plans and sequences of operations to achieve target objectives.
  4. **Execute**: Directs and schedules the specific interventions on the managed elements.
  - **Knowledge (K)**: Considered the "heart" of the architecture. It connects all four components together as a central repository, sharing models, logs, rules, and goals.
- **Touchpoints**: The standard interfaces that encapsulate a managed artifact.
  - **Sensors (Probes/Gauges)**: Export data regarding the resource's state and execution metrics.
  - **Effectors**: Provide the technical means to execute configuration or topological adjustments.
  - **Abstraction Purpose**: They allow the manager to monitor and influence complex resources without needing to know their internal implementation details.
- **Knowledge Types**:
  - **Innate (A Priori)**: Fixed knowledge engraved by software designers before execution (e.g., *"If CPU $>$ 80%, add a VM"*).

  - **By Acquaintance (Acquired)**: Dynamic information learned at runtime through sensors and environment interaction (e.g., *"There are 4200 users online"*).
- **Reasoning Approaches**:
  - **Rule-Based (Reflex)**: Stateless managers utilizing Event-Condition-Action (ECA) rules to quickly match events to plans. They are lightweight and fast but suffer from policy conflicts that are hard to detect before runtime.
  - **Model-Driven Autonomicity**: Uses synchronized runtime models (*models@runtime*) to separate data representations from execution logic. It maintains three independent models via *separation of concerns*: **Structure** (components and links), **Environment** (context resources), and **Non-Functional** (security/performance attributes). This lets models evolve without forcing a rewrite of the manager logic.
  - **Goal-Based vs. Utility-Based**: Goal systems classify states binary as desirable or undesirable. If an ideal state is blocked, they cannot choose the "least bad" option. Utility systems use a quantitative function to rank states mathematically, resolving conflicts when resource scarcity prevents meeting all goals simultaneously.
  - **Search-Based Reasoning**: Solves adaptation by finding an optimal path through a state space graph. It requires three items: **Initial and Target States**, a defined **Set of Actions** implemented via effectors, and a **Transition Model** mapping how actions trigger state transformations.
- **Architectural Models and Causal Connection**:
  - An architectural model represents software components as graphs of computational tasks and communication paths.
  - **Causal Connection**: A bidirectional, tight synchronization where changes in the running software instantly update the model, and changes verified on the model are immediately pushed to the running software. This connection stops the *stale state paradox*, preventing the manager from executing an adaptation based on outdated sensor data.
- **Probabilistic Reasoning**: Uses the Probability of Correctness (PoC) metric to deal with conflicting data from multiple Context Providers (CPs). Combining heterogeneous sensors (like a camera and pressure pad) filters out sensor noise and reduces uncertainty.
- **Learning Capabilities**: The most ambitious autonomic systems embed learning algorithms like reinforcement learning (resembling biological dopamine reward paths). This allows a generic manager to start with basic innate knowledge and discover sophisticated strategies tailored to its environment over time.

8.3.3 Self-Management Capabilities in Practice

- **Self-Optimization**: Constantly analyzing runtime metrics (CPU, response times) to balance conflicting objectives, such as satisfying Service Level Objectives (SLOs) while keeping resource costs minimal.
- **Self-Heal**: Monitoring application container lifecycles to detect instances lost to failure, and automatically launching replacement instances to keep system capacity above required thresholds.



9. FC3 - Autoscaling

9.1 The Auto-Scaling Process

9.1.1 MAPE Loop Phases

Identify and describe the four phases of the MAPE loop as applied to auto-scaling systems.:

- **Monitoring**: This phase is the system's eyes and ears. It collects real-time data from every level, including hardware (CPU and memory), the operating system, and the application (like response times). The goal is to make the auto-scaler able to know exactly how the system is performing against its Service Level Agreement (SLA);
- **Analysis**: Once the data is in, the analysis phase interprets it to determine if the system is healthy or if there can be a problem. This is where the system decides to be reactive (responding to problems happening right now) or proactive (predicting future traffic bursts). Proactivity is especially important because cloud resources don't appear instantly;
- **Planning**: This is the brain of the auto-scaler. It balances performance and cost to find the best solution. If traffic spikes, the planning phase decides whether to add resources or wait. Waiting helps avoid "oscillation"—a bad situation where the system scales up and down too quickly before seeing the effects of its changes;
- **Execution**: The final phase is the muscle of the loop. It uses the cloud provider's API to actually add or remove resources (like starting or stopping VMs). While this happens, the system must keep monitoring because new resources take time to become fully ready.

9.1.2 Proactive vs. Reactive Analysis

Compare reactive and proactive auto-scaling. Why is proactivity often necessary when dealing with sudden traffic bursts?: **Reactive auto-scaling** only responds to changes in the workload after they have been detected using the latest monitored variables. Instead **proactive auto-scaling** anticipates future demands to arrange resource provisioning in advance. Proactivity is especially important because acquiring new resources is not instantaneous. If the system is purely reactive, it will scale too late, leading to system congestion and SLA violations while waiting for new machines to boot up.

9.1.3 Monitoring Metrics

The paper lists several categories of monitoring metrics (Hardware, Load Balancer, Application Server, etc.). Why might an auto-scaler use a "proxy metric" like CPU utilization instead of direct application-level metrics?: An auto-scaler uses a “proxy metric” instead of direct application-level metrics because while the latter (like the average waiting time in a queue) can sometimes yield better scaling performance , proxy metrics, like CPU utilization, balance operational simplicity and reduced overhead and costs with a clear picture of the system’s current performance.

9.1.4 The "Boot-up" Challenge

How does the time required to boot and configure a new Virtual Machine (VM) impact the effectiveness of reactive auto-scaling?: The time required to boot a new VM severely limits reactive auto-scaling because there is always a delay from the time when an auto-scaling action is executed until it is effective. It takes several minutes to assign a physical server to deploy a VM, move the VM image to it, boot the operating system and application, and have the server fully operational.

9.1.5 Scaling Risks

Define the terms under-provisioning, over-provisioning, and oscillation in the context of resource allocation.:

- **Under-provisioning**: describe the state of an application that does not have the required resources to process incoming requests within the temporal limits of the SLA. This leads to system congestion and SLA violations, especially during sudden traffic bursts;
- **Over-provisioning**: describes the state of an application that maintains more resources than the necessary ones to comply with the SLA. While performance remains stable, the client will have unnecessary costs for idle or lightly loaded VMs;
- **Oscillation**: is the undesirable combination of both effects, occurring when scaling actions are executed too rapidly before the impact of previous actions can be measured. To avoid this, systems typically implement cooldown periods or capacity buffers to stabilize the resource environment.

9.2 Auto-Scaling Techniques & Classification

9.2.1 Threshold-Based Rules

- *Describe the basic structure of a threshold-based rule (the "condition" and "action"). What is the purpose of a "cooldown" or "calm" period in these systems?*
  A threshold-based rule is composed of 2 parts: a condition and an action to execute when the condition is met. The condition part uses one or more performance metrics. Each one has upper and lower

thresholds. If the condition is met for a given time then the corresponding action will be performed. After that, the autoscaler inhibits itself for a small cooldown period to avoid oscillations.

- *What could happen if, for a given performance metric, is set thrU=thrL?*
  If the upper and lower thresholds are set to the exact same value, the system does not have a tolerance zone. Any minor fluctuation in the workload will cause the metric to continuously cross the threshold up and down, triggering endless and immediate scale-up and scale-down actions. This leads to a severe state of oscillation, an undesirable effect that occurs when scaling actions are carried out too quickly, before being able to see the impact of the prevous scaling action.
- *Following the scale-out (in) rule principle, it is possible to define multiple scale-out (in) rule with thrU1 < thrU2 < . . . < thrUn (thrL1 > thrL2 > . . . > thrLn). For example, if the metric selected is the CPU utilization (CPU_Util) we can write the following rules:*
  If (CPU_util >= 60% and CPU_util < 70%) for durU sec then
  add 1 VM
  else If (CPU_util >= 70%) for durU sec then
  add 2 VMs
  Explain, why the above rules can be more effective of the following:
  If (CPU_utilization > 65%) for durU sec then add 2 VMs
  The multi-threshold policy is more effective because it introduces graduated reactions: small overload triggers a small scale-out, while larger overload triggers a larger one. This enables finer auto-scaling decisions, reduces the risk of over-provisioning, and may help avoid oscillations, compared with a single rule that always adds 2 VMs.
- *What Strategies for Triggering Auto-scaling Operations the condition "for durU sec then" implements?*
  The condition "for durU sec then" implements a persistence-based triggering strategy, where a threshold must be satisfied for a continuous time interval before scaling is applied. This helps filter transient workload fluctuations, reduces oscillations, and stabilizes reactive threshold-based auto-scaling decisions by ensuring actions are taken only under sustained system conditions.

9.2.2 Time Series Analysis (TSA)

What is the primary role of Time Series Analysis in an auto-scaling system, and what factors most heavily influence its prediction accuracy?: The primary role of Time Series Analysis in an auto-scaling system is to act as the main enabler of proactive auto-scaling techniques. Specifically, it is used in the analysis phase of the auto-scaling process, in order to find repeating patterns in the input workload or to try to forecast future values of metrics such as future workload or resource usage. Based on this forecasted value, a system can anticipate demands and a suitable auto-scaling action can be planned in advance. The main drawback of these techniques relies on their prediction accuracy, which highly depends on the target application, input workload pattern and/or burstiness, the selected metric, the history window and prediction interval, as well as on the specific technique being used. Among these factors, it is crucial to carefully select the algorithm and tune the parameters, specially the history window and the prediction interval to ensure that the forecast is reliable.

9.2.3 Control Theory (CT)

Explain the difference between "feedback" and "feed-forward" controllers. Why are they often combined in cloud environments?:

- **Feedback controllers** observe system outputs and are able to correct any deviation from the desired value;
- **Feed forward controllers** try to anticipate errors in the output. They predict, using a model, the behavior of the system and react before the error actually occurs.

The prediction may fail and, for this reason, feedback and feed-forward controllers are usually combined in cloud environments.

9.2.4 Queuing Theory (QT)

How is Queuing Theory used to estimate performance metrics like response time? What is a major limitation of using QT for general-purpose auto-scaling?: To estimate response time, Queuing Theory treats an application as a service center where incoming requests are measured against the processing speed of the available resources. During the Analysis phase of the MAPE loop, the auto-scaler calculates Utilization to determine how busy the servers are. As the system reaches its maximum capacity, the Queue Length and Waiting Time increase sharply. By adding the expected processing time to this calculated waiting time, the system predicts the total Response Time. This allows the auto-scaler to anticipate potential service agreement violations and trigger the Planning phase to add resources before the system becomes congested.
A major limitation of using Queuing Theory for general-purpose auto-scaling is its high sensitivity to model assumptions and "White-Box" requirements. Traditional models often assume steady, predictable patterns that may not reflect the "bursty" or unpredictable nature of real-world internet traffic. Furthermore, for a model to be accurate, the auto-scaler must precisely know the internal processing speed of the specific application. Because general-purpose cloud providers often treat applications as "black boxes," they lack the deep internal data needed to tune these complex mathematical models correctly, which can lead to Under-provisioning or Oscillation if the model's assumptions don't perfectly match the live workload.

9.2.5 Reinforcement Learning (RL)

Unlike Control Theory, Reinforcement Learning does not require a-priori knowledge of the system model. What is the primary disadvantage of this "trial-and-error" approach?: The primary disadvantage of this "trial-and-error" approach is its long learning phase, the bad initial performance and the lack of ability in react effectively to sudden bursts in the input workload.

9.2.6 Hybrid Techniques

Provide an example from the text of how two different techniques (e.g., TSA and Thresholds, or CT and RL) can be combined to create a more robust auto-scaler.: One of the hybrid examples presented in the paper is the combination of Reinforcement Learning (RL) and Control Theory (CT).
Reinforcement Learning (RL) is a type of automatic decision-making approach that has been used by several authors to implement auto-scalers. It operates without any a priori knowledge, learning the best actions through a trial-and-error approach based on rewards.
Control Theory (CT) relies on a controller whose main objective is to automate the management (e.g. scaling task) of a target system. It does this by attempting to maintain the value of a controlled variable y (e.g. CPU load) close to the desired level or set point y_{ref} by adjusting the manipulated variable u (e.g. number of VMs).
The example proposed in the paper highlights a specific implementation by Zhu et al. that successfully merges these two techniques. In their approach:

- They use a standard CT tool called PID (Proportional Integral Derivative) controller, but they integrate an RL agent specifically to estimate the derivative term of this controller;
- Through trial-and-error training, the RL agent learns to minimize the sum of the squared error of the control variables (the adaptive parameters) without violating the time and budget constraints over time;
- This hybrid controller guides the adaptation of application parameters (such as image size) to meet the Service Level Agreement (SLA);
- Consequently, virtual resources like CPU and memory are dynamically provisioned based on these adaptive changes.

9.3 Evaluation and Experimental Methods

9.3.1 Simulation vs. Real Platforms

What are the advantages of using a cloud simulator (like CloudSim) over testing on a real public cloud provider for auto-scaling research?: Cloud simulators offer major advantages for auto-scaling research, including low cost, full control over workload and system parameters, reproducibility and the ability to test extreme or hypothetical scenarios quickly and safely. In contrast, real cloud platforms introduce cost, variability and limited control, making large-scale and repeatable experimentation more difficult, although they provide more realistic evaluation conditions.

9.3.2 Workload Traces

The paper mentions several real-world traces used for testing, such as the "World Cup 98" and "ClarkNet" traces. What is a key benefit of using real traces compared to purely synthetic workloads?: Using real workload traces provides the key benefit of realism, analyzing actual traffic patterns, variability and burstiness that synthetic workloads often fail to represent. This leads to a more reliable and meaningful evaluation, since autoscaling algorithms are tested under conditions that closely reflect real-world behaviour, including unpredictable spikes and temporal correlations that are difficult to model synthetically.

9.3.3 Application Benchmarks

Name two commonly used benchmarks for cloud research mentioned in the Appendix and describe the type of application they simulate.: Two commonly used benchmarks are:

- **RUBiS**, that simulates an online auction application similar to eBay with workloads involving browsing, bidding and item management, and it is useful for evaluating systems with read-write interactions and user concurrency;
- **TPC-W**, which simulates an e-commerce web application similar to an online bookstore, including browsing, searching and purchasing, and it represents transactional workloads with a mix of read and write operations.

Both benchmarks are valuable because they model realistic multi-tier web applications, providing the possibility to evaluate autoscaling under interactive user-driven workloads, typical in cloud environments.

9.4 Evaluating Auto-scaling Strategies for Cloud Computing Environments

9.4.1 Adaptive Strategy for Setting Autoscaling Step Size

What problem in threshold-based autoscaling the "adaptive strategy for setting autoscaling step size" aims to solve?: The adaptive strategy for setting the autoscaling step size aims to overcome the limitations of fixed step sizes in threshold-based autoscaling, which can cause slow reactions or overshooting. It dynamically adjusts the step size based on current utilisation to bring the system directly within the target interval [L,U], reducing inefficiency.

9.4.2 Adaptive Step Size

Does the multiple thresholds rules in 6.c implement a sort-of adaptive step size?: Yes, the multiple thresholds rules in 6.c implement an adaptive step size because in this scenario we're using a strategy that, in each time-step t, computes a step size according to the current utilisation level. In particular if the CPU utilization is between 60 and 70% we are adding only 1 additional VM, while if the CPU utilization is larger than 70% we are adding 2 additional VM.

9.4.3 Conservativeness vs. Reactiveness

Strategies for Triggering Auto-scaling Operations could be Reactive, Conservative and Predictive. What are the benefits of conservativeness over reactiveness?: Strategies for Triggering Auto-scaling Operations could be:

- **Reactive**, when as soon as the utilization exceeds the range [L, U] we auto-scale the resources;
- **Conservative**, when we auto-scale the resources if the utilization exceeds the range [L, U] for 4 consecutives times t;
- **Predictive**, when we predict that at the following time t+1 the utilization will exceed the range [L, U].

The benefit of conservativeness over reactiveness is that in scenarios where the CPU utilization has often peaks but generally it is in the range [L, U] with the reactiveness we will auto-scale the resources uselessly.

9.4.4 Autoscaling Demand Index

What is the aim of the Autoscaling Demand Index metric? What is the advantage of using the ADI metric over other performance metrics to evaluate the effectiveness of an autoscaling algorithm?: The Auto-scaling Demand Index (ADI) is introduced to measure how well an autoscaling strategy keeps system utilisation within desired target interval [L,U], quantifying how far and how often the system operates outside the acceptable range. The main advantage of using ADI is that it provides a unified and balanced evaluation of autoscaling performance: it captures both types of inefficiency and it measures the magnitude and duration of these deviations.

9.4.5 Use Case

Let assume that the desired upper and lower bound (L, U) for CPU utilization are L=50, U=75. Let assume also that the number of VMs allocated is mti = 4 and we have the following sequence of CPU utilization observations (the sequence is an aggregated value, e.g., average, over the four VMs).

$u_{t1} = 40, u_{t2} = 66, u_{t3} = 70, u_{t4} = 85, u_{t5} = 90, u_{t6} = 95$

Determine the step scaling size for each time ti, both for an aggressive and conservative strategy. Moreover

determine the value for an aggressiveness degree of 0.7.: Using the following equations we can find the adaptive step size:

$Scale-in: st = (1-\alpha)L_t + \alpha U_t$

$Scale-out: st = \alpha L_t + (1-\alpha)U_t$

$L_t = mt * (L-u_t)/L$

$U_t = mt * (U-u_t)/U$

t	Utilization	Action	Step size (α = 1)	Step size (α = 1)	Step size (α = 0.7)
1	0.40	Scale-in	2	1	2
2	0.66	None	0	0	0
3	0.70	None	0	0	0
4	0.85	Scale-out	1	3	2
5	0.90	Scale-out	1	4	2
6	0.95	Scale-out	2	4	2

9.5 Sum-up

9.6 Chapter Summary and Exam Preparation Recap

9.6.1 1. Fundamental Concepts & The MAPE Loop

- **Cloud Elasticity:** The ability to dynamically provision and deprovision computing resources (usually Virtual Machines) to match variable workloads in a pay-as-you-go economic model.
- **The MAPE Loop:** The foundational architectural framework for autonomic auto-scaling systems, divided into four sequential phases:
  - **Monitoring (M):** Gathers real-time performance metrics across multiple domains (Hardware: CPU/RAM; Load Balancer: queue length/sessions; Application: response time). It often uses hypervisor-level proxy metrics (e.g., CPU utilization) to minimize overhead and operational complexity.
  - **Analysis (A):** Evaluates the monitored data to determine system state or predict future demands. It can be *reactive* (responding to past/current data) or *proactive* (forecasting traffic bursts).
  - **Planning (P):** Acts as the decision-making core. It processes analytical insights against target Service Level Agreements (SLAs) to decide whether to alter the resource pool while balancing performance and cost.
  - **Execution (E):** Dispatches commands via the cloud provider's API to physically add or remove instances.
- **Core Resource Allocation Risks:**
  - **Under-provisioning:** The application lacks sufficient resources to satisfy demand, directly causing system congestion and SLA violations.
  - **Over-provisioning:** Excess resources are leased beyond what is required to satisfy the SLA, leading to idle capacity and unnecessary financial costs.
  - **Oscillation (Flapping):** Rapid, consecutive scale-out and scale-in actions caused by changing capacity before the impact of previous actions can stabilize.

  - **The Boot-up Delay Challenge:** Virtual Machines require several minutes to allocate hardware, copy images, boot the OS, and fully initialize applications. This latency restricts the effectiveness of purely reactive scaling during sudden traffic spikes.

9.6.2 2. Taxonomy of Auto-Scaling Techniques

- **Threshold-Based Rules:**
  - **Mechanism:** Executes predefined condition-action pairs (e.g., if CPU $>$ thrU for durU seconds, then add s VMs).
  - **Triggering Strategies:** Employs a persistence-based triggering strategy via the duration parameter (durU) to filter out brief, transient noise.
  - **Stabilization:** Implements *cooldown/calm periods* during which the auto-scaler inhibits further modifications, allowing newly added systems to boot and absorb load.
  - **Multi-Threshold Rules:** Utilizing graduated thresholds (e.g., separate rules for 60% and 70% load) allows for multi-step adaptive reactions, decreasing the risk of overshooting and oscillation compared to single-threshold designs. Setting thrU $=$ thrL destroys the safety buffer zone, inducing chronic oscillation.
- **Time Series Analysis (TSA):**
  - **Mechanism:** The primary driver for proactive auto-scaling. It analyzes uniform historical data points to extract repeating patterns (trend, seasonality, cycles) or directly forecast future metrics.
  - **Methods:** Ranging from simple Moving Averages (MA) and Exponential Smoothing (ES) to Auto-Regressive models (AR(p) / ARMA / ARIMA) and Machine Learning models (Neural Networks, Linear Regression).
  - **Parameters:** Accuracy heavily depends on tuning the *history window* (sensitivity to long vs. short-term patterns) and the *prediction interval* (which should match or exceed the VM boot-up time).
- **Control Theory (CT):**
  - **Mechanism:** Treats the auto-scaler as a dynamic controller designed to manage a target output variable (e.g., resource utilization) close to a reference set-point (y_{ref}) by continuously adjusting a manipulated input variable (u, such as VM count).
  - **Architectures:**
    - **Feedback Controllers:** Reactive loops that measure current system output error and dynamically apply mathematical corrections (e.g., standard Proportional-Integral-Derivative (PID) laws).
    - **Feed-forward Controllers:** Proactive loops that predict system disturbances using a structural model and apply resource modifications before errors manifest.
  - **Advanced Designs:** Combine both feedback and feed-forward elements to overcome model prediction errors. Implementations include Adaptive Controllers (on-line parameter tuning via ARMAX or Kalman filters), Fuzzy Controllers (rule-based reasoning mapping inputs to continuous membership sets), and Model Predictive Control (MPC).
- **Queuing Theory (QT):**
  - **Mechanism:** Analytical white-box performance modeling that treats cloud applications or load balancers as mathematical waiting lines (e.g., $M/M/1$, $G/G/n$ queues).

  - **Metrics Allocation:** Relies on Little's Law ($\mathbb{E}[C] = \lambda \times \mathbb{E}[T]$) and arrival/service rates (λ, μ) to estimate internal queues, waiting times, and total end-to-end response times to compute required node counts.
  - **Limitation:** Highly rigid and sensitive to model assumptions. It struggles with bursty, non-stationary traffic, and requires precise internal parameters of the application, rendering it less effective for general-purpose "black-box" cloud deployments.
- **Reinforcement Learning (RL):**
  - **Mechanism:** A model-free, experience-driven approach based on direct interaction. An agent maps system states (S) to scaling actions (A) online, driven by a mathematical reward function (R) designed to optimize costs and minimize SLA violations.
  - **Algorithms:** Primarily uses Q-learning or SARSA to update a cumulative value table or function approximation (via Neural Networks or CMAC).
  - **Limitation:** Suffers from a long online learning convergence phase, poor early-stage random performance ("curse of dimensionality"), and difficulty adapting to sudden workload shocks due to exploration actions.
- **Hybrid Systems:** Combine complementary architectures to produce more robust auto-scalers. An example includes blending Control Theory with Reinforcement Learning, where a standard PID controller manages virtual resources, while an RL agent runs simultaneously to estimate the derivative control term online to minimize error parameters without violating budget limitations.

9.6.3 3. Experimental Evaluation Frameworks

- **Evaluation Platforms:**
  - **Simulators (e.g., CloudSim, GroudSim):** Discrete event software tools. They offer low cost, highly accelerated execution speeds, perfect environment isolation, full metric visibility, and absolute parameter reproducibility, though they remain structural abstractions of real hardware.
  - **Custom Testbeds / Public Clouds:** Physical infrastructure deployments utilizing hypervisors (Xen, KVM, ESXi) managed by cloud orchestration stacks (OpenStack, Eucalyptus). They feature high engineering setup complexity and high execution costs, but capture true environmental variance, OS overhead, and actual hardware interference.
- **Application Benchmarks:** Realistic tiered packages used to drive repeatable load during scaling experiments.
  - **RUBiS:** Simulates an interactive auction platform modeled after eBay, tracking concurrent user profiles across web, application, and database tiers.
  - **TPC-W:** An e-commerce bookstore prototype measuring transactional web interactions per second over transactional read/write mixes.
  - **WikiBench:** A performance testing framework that utilizes MediaWiki engines driven by real historical data and traffic traces from Wikipedia.
- **Workload Characteristics:**
  - **Synthetic Workloads:** Artificially generated request streams configured around mathematical profiles (Stable, Growing, Cyclic/Bursting, On-and-off) using specialized tools (Faban, JMeter, Httperf, Rain).
  - **Real Traces:** Production server logs (e.g., ClarkNet cyclic ISP records, World Cup 98 logs, and

Google Cluster Data sets) that capture natural human variations, historical correlations, and sharp burst spikes.

9.6.4 4. Scaling Step Size Strategies & The ADI Metric

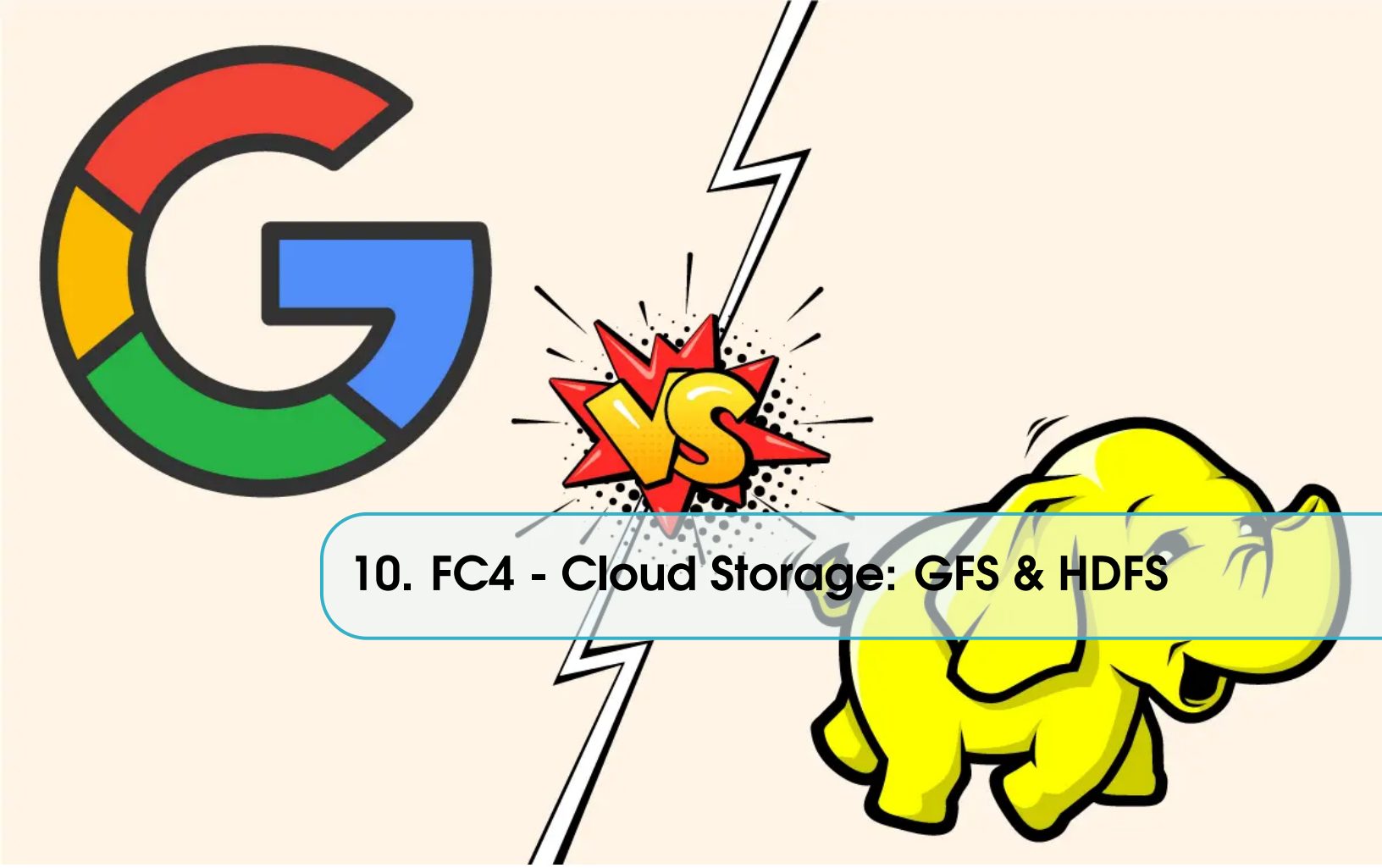
- **The Auto-Scaling Demand Index (ADI):** A comprehensive performance metric (σ) that evaluates the quality of an auto-scaling strategy by penalizing any deviation of the actual system utilization (u_t) from the target interval bounds ($[L,U]$):

$$\sigma = \sum_{t \in \mathcal{T}} \sigma_t \quad \text{where} \quad \sigma_t = \begin{cases} L - u_t & \text{if } u_t \leq L \\ 0 & \text{if } L < u_t < U \\ u_t - U & \text{otherwise} \end{cases}$$

  ADI provides a balanced evaluation by using the same unit to penalize both poor Quality of Service (under-provisioning) and excess infrastructure cost (underutilization).
- **Triggering Strategies Comparison:**
  - **Reactive:** Evaluates instantaneous metrics (u_t). It achieves low ADI by reacting immediately to departures from the target interval.
  - **Conservative:** Evaluates multi-step window sequences (e.g., requiring 4 consecutive time-step violations). It smooths out narrow transient peaks but incurs higher ADI penalties due to its slow, delayed response.
  - **Predictive:** Estimates upcoming load (u'_{t+1}) using smoothing or machine learning. It reduces ADI on steady workloads, but poor forecasts on irregular data can trigger incorrect resource modifications.
- **Resource Step-Size Modifications (s_t):**
  - **Fixed Step Size:** Uses a constant resource block percentage. It is effective if the underlying traffic is stable, but requires extensive historical workload knowledge to tune. If the interval $[L,U]$ is narrow, large fixed steps cause frequent overshooting across the boundaries.
  - **Adaptive Step Size:** Dynamically calculates resource requirements at each time-step based on the distance to the target interval bounds. It computes operational boundary conditions (L_t, U_t) and uses an aggressiveness parameter ($\alpha \in [0,1]$) to balance resource changes:

$$\text{Scale-in:} \quad s_t = (1-\alpha)L_t + \alpha U_t \qquad \text{Scale-out:} \quad s_t = \alpha L_t + (1-\alpha)U_t$$

  Adaptive scaling reduces ADI under bursty and peaky loads by matching resource capacity to demand in fewer iterations, preventing long convergence delays.



10. FC4 - Cloud Storage: GFS & HDFS

10.1 Task 1 - GFS

10.1.1 Q1.1 GFS uses HeartBeat for Master Check Server communication. Why are rather than a synchronous request-response interaction? What information is transferred via messages, and how is it used?

GFS uses the HeartBeat approach instead of synchronous request-response interactions to avoid making the master a performance bottleneck. Because the master manages thousands of chunkservers simultaneously, actively polling each one would create massive overhead and reduce its availability for client operations. HeartBeat messages allow the master to collect updates passively and batch communications efficiently. These messages transfer two main types of information:

- **State reports**: Sent from chunkservers to the master to report their status and data chunks;
- **Instructions**: Sent from the master back to chunkservers to assign tasks (e.g., data replication or cleanup).

The master uses this information to monitor chunkserver liveness, detect failures and maintain accurate records of chunk replicas across the cluster.

10.1.2 Q1.2 What are the advantages of having a chunk size of 64 MB?

There are several advantages in using large chunk size of 64 MB:

- It drastically reduces how often clients need to interact with the master server. Reading and writing data on the same chunk requires only one initial request to the master to get the chunk location information. Once the client has this info, it interacts directly with the chunkservers for subsequent operations;

- It minimizes network communication overhead. Because a chunk is so large, a client is highly likely to perform many consecutive operations on that specific chunk. This allows the client to maintain a persistent TCP connection to the chunkserver over an extended period of time rather than constantly opening new ones;
- It significantly reduces the total size of the metadata that the master server needs to track since there are fewer chunks.

10.1.3 Q1.3 What is the technique used to reduce the interactions between the client and the master when a file is accessed in read mode?

The primary technique used to reduce interaction between the client and the master is caching. Clients never read and write file data through the master. Instead, a client asks the master which chunkservers it should contact. Once the master replies with the chunk handle and replica locations the client caches this information using the file name and chunk index as the key. Further reads of the same chunk require no more client-master interaction until the cached information expires or the file is reopened.

10.1.4 Q1.4 How does the Master obtain the information about the chunk location?

At Startup the Master does not keep a persistent record of chunk locations. Instead, it simply polls all the chunkservers to ask them what chunks they currently have.
After Startup the Master keeps its information up-to-date because it controls all new chunk placements and continuously monitors the status of the chunkservers using regular HeartBeat messages.

10.1.5 Q1.5 Why is the Operation Log central to GFS, and how does the Master use the information stored in the Operation Log?

The Operation Log is central to GFS because it is the sole persistent record of critical metadata and serves as a logical timeline that defines the exact order of concurrent operations, uniquely identifying files and chunks by their creation time. The Master uses this log primarily to recover its file system state by replaying historical operations. To ensure reliability, the Master batches and flushes log records to both local and remote disks before confirming any changes or responding to client operations. Additionally, to minimize startup and recovery time, the Master periodically compacts the log into memory-mappable checkpoints, allowing it to quickly load the latest checkpoint and only replay the small number of log records that occurred afterward.

10.1.6 Q1.6 What is the meaning of defined, undefined, consistent, and inconsistent regions for a file in GFS? When a file is in one of the above-mentioned states, how does the GFS recognize a given state?

In GFS the state of a file region after a data mutation depends on the type of mutation, whether it succeeds or fails and whether there are concurrent mutation. A region can be in one of the following states:

- **Consistent**: A region is consistent if all clients will always see the same data, regardless of which replicas they read from;
- **Defined**: A region is defined if it is consistent and clients will see what the mutation writes in its entirety. This happens when a mutation succeeds without interference from concurrent writers;
- **Inconsistent**: A region is inconsistent if the mutation is failed. Data will not be consistent since different clients may read different data in different moments or on different replica;

- **Undefined**: A region is undefined if all clients will see the same data but these data are mixed fragments that come from different writes.

GFS applies a set of mechanisms in order to force or detect these status:

- GFS ensures that the mutation are applied in the same exact order on all the replica: the primary replica assigns consecutive serial numbers to mutations, providing the necessary serialization;
- The Master maintains a version number of the chunks that is incremented at each new lease. If a replica was offline during a mutation, its version number will be smaller than the others. In this case the master will exclude it from the client requests, considering it as not existing and then deleting it;
- the Master uses regular handshake messages in order to verify if the chunkservers are alive. The latter utilize checksums in order to autonomously detect if data are corrupt or are inconsistent, signaling the error to the Master.

10.1.7 Q1.7 How does GFS maintain a consistent mutation order across replicas?

GFS maintains a consistent mutation order across replicas primarily through a lease mechanism. One of the replicas is designating as the "primary" by the Master Node. This primary replica is responsible for assigning consecutive serial numbers to all incoming mutations, establishing a strict serial order. After the client pushes the data to all replicas, the primary forwards the write request to the secondary replicas, which are required to apply the mutations to their local state in the exact same serial number order dictated by the primary. Therefore, the global mutation order is defined first by the master's sequence of lease grants, and subsequently by the specific serial numbers assigned by the primary within its lease term. To further ensure consistency, GFS uses chunk version numbers to detect and isolate any stale replicas that may have missed mutations due to downtime, preventing them from serving outdated data or participating in new mutations.

10.1.8 Q1.8 What is the strategy used by GFS to transfer data to replicas, avoiding network bottlenecks and high-latency links, and to minimize latency?

GFS optimizes data transfer to replicas by decoupling the data flow from the control flow. To avoid network bottlenecks and high-latency links, GFS pushes data linearly along a carefully selected chain of chunkservers rather than using a tree topology. Within this chain, each machine forwards the data to the "closest" remaining machine based on network topology distances, which are estimated using IP addresses. Furthermore to minimize latency, as soon as a chunkserver receives a portion of the data, it immediately begins forwarding it to the next machine in the chain without waiting for the entire transfer to complete.

10.1.9 Q1.9 Why is it not important that all the GFS replicas are byte-wise identical?

Byte-wise identity across GFS replicas is not important because GFS uses a relaxed consistency model where divergent replicas may be legal. Due to client retries after failures, replicas of the same chunk may contain different data possibly including duplicates. GFS only guarantees that the data is written at least once as an atomic unit. This lack of byte-wise identity does not cause problems because applications are built to handle these inconsistencies by design. To manage this, a reader can identify and discard extra padding and record fragments using the checksums. Furthermore, if an application cannot tolerate duplicate records, it can filter them out using unique identifiers in the records.

10.1.10 Q1.10 What are the goals of the Replica Placement policy used by GFS? How are the intended goals achieved?

The goals of the GFS Replica Placement policy are to maximize data reliability and availability, and to maximize network bandwidth utilization. GFS achieves these goals by:

- Spreading replicas across racks ensures that data survives and remains available even if an entire rack is damaged or goes offline. This protects the system against the failure of a shared resource, such as a rack's network switch or power circuit;
- This multi-rack distribution allows network traffic—especially read operations—to exploit the aggregate bandwidth of multiple racks simultaneously.

10.1.11 Q1.11 How did the master choose where to place a newly created chunk? When and why does the muster re-replicate a chunk?

When the master creates a chunk, it chooses where to place the initially empty replicas. It considers several factors:

- We want to place new replicas on chunkservers with below-average disk space utilization. Over time this will equalize disk utilization across chunkservers;
- We want to limit the number of "recent" creations on each chunkserver. While creating a chunk is cheap, it often signals imminent heavy write traffic, as chunks are typically created because of a write request.
- We want to spread replicas of a chunk across racks. This ensures data availability even if an entire rack's shared resource, such as a network switch or power circuit, fails.

The master re-replicates a chunk as soon as the number of available replicas falls below a user-specified goal (3 by default). This could happen for various reasons:

- a chunkserver becomes unavailable;
- it reports that its replica may be corrupted;
- one of its disks is disabled because of errors;
- the replication goal is increased.

10.1.12 Q1.12 How is the master state replicated for reliability? Why are " shadow " master replicas preferred to "mirror" replicas?

To ensure the system remains highly available and reliable, the master state is replicated using the following strategies:

- **Log and Checkpoint Replication**: the operation log and checkpoints are replicated on multiple remote machines;
- **Strict Commitment Rule**: a state mutation is only considered committed after its corresponding log record has been flushed to disk both locally and on all remote master replicas.
- **Fast Restart and Failover**: if the master process fails, it can restart almost instantly using the replicated operation log;
- **Canonical DNS Alias**: clients interact with the master using a canonical name (e.g., gfs-test), which is a DNS alias. If the master needs to be relocated to a different machine after a failure, this alias can simply be changed to point to the new machine seamlessly.

GFS prefers shadow masters over mirror replicas to maintain design simplicity. Instead of using complex,

synchronous replication to stay perfectly in sync, a shadow master updates itself passively by reading a replica of the operation log, meaning it may lag behind the primary master by fractions of a second. This slight delay is an acceptable tradeoff because actual file content is read directly from chunkservers; therefore, shadow masters can safely provide high read-only availability for metadata even when the primary master is offline.

10.2 Task 2 - HDFS

10.2.1 Q2.1 Explain the concept of image, checkpoint, and journal in HDFS

The image represents the complete state of the file system metadata at a specific point in time. It consists of inodes data, which record files and directories attributes and the list of blocks belonging to each file. The checkpoint is the persistent record of the namespace image that is stored in the local host's native file system. The journal is a persistent, write-ahead commit log that tracks every change made to the file system metadata.

10.2.2 Q2.2. During normal operating conditions, DataNode sends heartbeat messages to the NameNode. Which information do these heartbeats carry? How does the NameNode communicate with the DataNode?

Heartbeats from a DataNode carry informations about total storage capacity, fraction of storage in use and the number of data transfers currently in progress. These statistics are used for the NameNode's space allocation and load balancing decisions. The NameNode does not directly call DataNodes. It uses replies to heartbeats to send instructions to the DataNodes.

10.2.3 Q2.3 When and why do DataNode and NameNode perform a handshake? What happens after a handshake?

During startup each DataNode connects to the NameNode and performs a handshake. The purpose of the handshake is to verify the namespace ID and the software version of the DataNode. If either does not match that of the NameNode, the DataNode automatically shuts down. After the handshake the DataNode registers with the NameNode. DataNodes persistently store their unique storage IDs. The storage ID is an internal identifier of the DataNode, which makes it recognizable even if it is restarted with a different IP address or port.

10.2.4 Q2.4 Can HDFS store and handle files with different replication factors? (E.g., file1 with replication factor 3 and file2 with replication factor 5)

Yes, HDFS can store and handle files with different replication factors. HDFS provides an API that allows an application to set the replication factor of a file. By default a file's replication factor is three. For critical files or files which are accessed very often, having a higher replication factor improves their tolerance against faults and increase their read bandwidth.

10.2.5 Q2.5 How can a client application verify if a data block is corrupted? Is data corruption verified only by the client at read time?

When a client creates an HDFS file, it computes a checksum sequence for each block and sends it and the data to a DataNode, which stores these checksums in a separate metadata file, distinct from the block's actual data file. Then, when HDFS reads a file, both the block data and its checksums are shipped to the client,

which recomputes the checksum for the received data and compares it against what it received; so, if there's a mismatch, the client notifies the NameNode of the corrupt replica and fetches a different replica from another DataNode. Moreover, data corruption is not verified only at read time, since each DataNode runs a block scanner that periodically and independently scans all its block replicas, verifying that stored checksums match the block data.

10.2.6 Q2.6 What are the recommended practices for storing the checkpoint and journal?

The recommended practices are:

- Storing in multiple storage directories, since the NameNode should be configured to write both the checkpoint and journal to more than one storage directory and if the NameNode encounters an error writing the journal to one directory it automatically excludes that directory and continues with the others;
- Placing directories on different volumes, since spreading storage directories across separate physical volumes protects against single-volume failures;
- Placing one storage directory on a remote NFS server, since this allows to protect against failure of the entire NameNode host machine not just a single disk;
- Creating checkpoints daily, since a very large journal increases both the risk of corruption and the NameNode restart time.

10.2.7 Q2.7 Why does the CheckpointNode periodically combine the existing checkpoint and journal to create a new checkpoint and an empty journal?

The CheckpointNode periodically merges the existing checkpoint and journal to protect file system metadata and enable system recovery if other persistent copies become unavailable. Crucially, this process allows the NameNode to truncate the constantly growing journal. By preventing the journal from becoming infinitely large, the system minimizes the risk of journal file loss or corruption and significantly reduces the time it takes to restart the NameNode.

10.2.8 Q2.8 Does HDFS allow random writes on a file? (A random write is when a client application writes at a specified offset in a file)

HDFS does not allow random writes (writes at a specified offset in a file) because it is designed around a model where files are created and written once. After a file is closed, the bytes already written cannot be altered or removed. The only operation allowed is appending something at the end of the file.

10.2.9 Q2.9 What are the steps performed by a client application and a DataNode for writing to a file? Can multiple clients write simultaneously on a file?

An HDFS file consists of blocks. When there is a need for a new block, the NameNode allocates a block with a unique block ID and determines a list of DataNodes to host replicas of the block. The DataNodes form a pipeline, the order of which minimizes the total network distance from the client to the last DataNode. Bytes are pushed to the pipeline as a sequence of packets. The bytes that an application writes first buffer at the client side. After a packet buffer is filled (typically 64 KB), the data are pushed to the pipeline. The next packet can be pushed to the pipeline before receiving the acknowledgement for the previous packets. The number of outstanding packets is limited by the outstanding packets window size of the client. After data are

written to an HDFS file, HDFS does not provide any guarantee that data are visible to a new reader until the file is closed. The HDFS client that opens a file for writing is granted a lease for the file; no other client can write to the file. The writing client periodically renews the lease by sending a heartbeat to the NameNode. When the file is closed, the lease is revoked.

10.2.10 Q2.10 What are the steps performed by a client application and a DataNode for reading a file? Can multiple clients read a file simultaneously? Are concurrent read and write operations on a file permitted?

To read a file in HDFS, the client application must first locate the data from the central directory. When an application reads a file, the HDFS client first asks the NameNode for the list of DataNodes that host replicas of the blocks of the file. To optimize network bandwidth the locations of each block are ordered by their distance from the reader. Once the location is known the client tries the closest replica first and contacts a DataNode directly and requests the transfer of the desired block. If for some reason the target DataNode is unavailable or the data is corrupt, the read attempt fails, the client tries the next replica in sequence.
A file may have many concurrent readers. The HDFS architecture is specifically designed around a "single-writer, multiple-reader model". Because of this a writer's exclusive access does not prevent other clients from reading the file.
Furthermore the system permits a client to read a file that is open for writing. However, when reading a file open for writing, the length of the last block still being written is unknown to the NameNode. In this case, the client asks one of the replicas for the latest length before starting to read its content.

10.2.11 Q2.11 How are the DataNodes selected and ordered for a write operation? How are the DataNodes (replica locations) selected and ordered for a read operation?

For a write operation the default policy is that the NameNode chooses three DataNodes: it places the first replica on the application node (writer), while the second and third are placed on different nodes within a different rack. The selected nodes are organized in an ordered pipeline such that we minimize the total network distance between the client and the last DataNode.

10.2.12 Q2.12 Describe the default HDFS block placement policy and the rationale behind the placement decisions.

The default HDFS block placement policy places the first replica on the node where the writer is located, and the second and third replicas on two different nodes within a different rack. Any additional replicas are placed randomly across the cluster, strictly ensuring that no single node holds more than one replica of a block, and no single rack holds more than two replicas (provided there are enough racks). The rationale behind this policy is to balance minimizing write costs with maximizing data reliability, availability, and aggregate read bandwidth. Placing the second and third replicas on a separate rack better distributes the data while reducing expensive inter-rack write traffic, which improves write performance. Because rack failures are significantly less common than individual node failures, this setup maintains strong reliability guarantees while also reducing the aggregate network bandwidth used during read operations, as the standard three replicas only span two unique racks instead of three.

10.2.13 Q2.13 How does HDFS guarantee that the disk utilization is balanced across the cluster Datanodes?ca

HDFS does not inherently guarantee balanced disk utilization during initial block placement, as doing so could cluster new, highly referenced data on a few empty nodes. Instead, balancing is achieved using a separate administrative tool called the Balancer. The Balancer iteratively moves replicas from DataNodes with high disk utilization to those with lower utilization. A cluster achieves a balanced state when the disk utilization of every individual DataNode differs from the overall cluster's average utilization by no more than a configured threshold fraction. While moving data, the Balancer strictly maintains data availability by never reducing the total number of replicas or racks. It also optimizes network traffic by prioritizing intra-rack data copying and allows administrators to limit the bandwidth consumed by rebalancing operations to avoid competing with standard application processes

10.3 Task 3 - Similarities and Differencies between GFS and HDFS

GFS and HDFS have the following similarities and differences:
Interactions among the client application, the node storing the data (data node and chunk server), and the master (or Name) node
When we consider the interaction of the client application we can see this main differences:

- GFS: The client sends the file name and chunk index to the master. The master replies with the chunk handle and locations of replicas. The client then caches this information and communicates directly with the chunkservers.
- HDFS: The client contacts the NameNode to get the locations of data blocks. It then reads directly from the DataNode closest to it. Instead, when we talk about how data are stored inside the nodes:
- GFS: files are collected in fixed-size segments called chunks of size 64MB. They use Linux File system. Replicated on multiple sites (3 default but they are configurable)
- HDFS: files are collected in blocks of size 64-128MB. And even here data are collected in at least 3 replicas. In order to apply a change to a file, there are two different mechanisms:
- GFS: The client send the modification to each replica of the file;
- HDFS: The client sends the modification only to the primary replica, and then it will be forwarded to the other one on cascade.

Communication strategy/technique used by the master/name node to interact with the nodes storing the data
Both GFS and HDFS adopt an architecture with a single master node (MasterNode in GFS and NameNode in HDFS) in order to simplify the design and allow simple centralized decisions about the data replication. The central node in both systems relies on periodic heartbeat messages sent from storage nodes to monitor their health and availability. Neither the HDFS NameNode nor the GFS MasterNode stores the physical locations of block replicas in a persistent record, preferring to reconstruct this mapping from reports sent by storage nodes at startup. The master nodes do not initiate direct calls to storage nodes but instead deliver instructions by piggybacking them onto the responses of received heartbeats.
One notable difference is that HDFS requires a formal handshake during startup where DataNodes must verify their namespace ID and software version with the NameNode. HDFS DataNodes provide the NameNode with a complete view of all hosted replicas by sending comprehensive block reports every hour. In GFS, chunkservers instead report only a subset of their chunks within the regular heartbeat messages exchanged

with the master. The GFS MasterNode specifically uses the heartbeat mechanism to grant and extend chunk leases to primary replicas, delegating the sequencing of mutations to those nodes. When heartbeats cease, both systems eventually consider the storage node to be out of service and automatically schedule the creation of new replicas on other available nodes.

Data replication strategies and data placement strategies

Both systems store 3 copies of each piece of data, spread across different racks so data is safe even if a rack fails. When some copies are missing, the system gives priority to the most under-replicated data. The master (or NameNode) only monitors and tells data nodes what to do, while the actual data copying happens directly between nodes.

There a few differences between the two systems:

- replica placement: in GFS, placement is flexible and decided by the master each time, balancing space and distribution; in HDFS, there is a fixed rule, where the first copy goes on the writer's node and then other copies go on different nodes in another rack, reducing network traffic while still keeping data safe.
- load balancing: while GFS does it automatically in the background, HDFS uses a separate tool called the balancer that must be run manually.
- detecting outdated replicas: while GFS uses version numbers to detect if a node missed updates, HDFS uses a similar concept called "generation stamps".
- removing extra replicas: both remove extra copies, but HDFS is more specific, since it removes replicas from nodes with the least free space, while still keeping data spread across racks.

Access modes (In which modes client applications can write/read data)

Both GFS and HDFS are optimized for append-only mutations and large sequential reads. In GFS, most files are mutated by appending new data rather than overwriting existing data and once written, the files are only read, and often only sequentially. Similarly, HDFS dictates that after the file is closed, the bytes written cannot be altered or removed except that new data can be added to the file by reopening the file for append. Furthermore both systems support concurrent reading during active writes. The most significant difference is their handling of concurrent writes. GFS natively supports multiple simultaneous writers through a record append operation that allows multiple clients to append data to the same file concurrently while guaranteeing the atomicity of each individual client's append. Instead HDFS strictly forbids this because it implements a single writer, multiple-reader model meaning once a writer opens a file no other clients can write to the file. Moreover, while HDFS strictly prevents altering previously written bytes, GFS technically allows it (Small writes at arbitrary positions in a file are supported but do not have to be efficient)

Data integrity, high availability, and recovery from failures

To ensure data integrity, GFS maintains 32-bit checksums for every 64KB block within a chunk, with chunkservers performing verification during reads and idle periods. Similarly, HDFS stores checksums in separate metadata files, though it places more emphasis on client-side verification and utilizes a background Block Scanner to identify corruption. For high availability, both systems record metadata changes in operation logs and use periodic checkpoints. GFS provides "shadow masters" to allow read-only access if the primary master fails, while HDFS utilizes BackupNodes to maintain a synchronized image of the namespace for faster recovery. Failure recovery in both systems relies on heartbeats to monitor node health; once a failure is detected, the systems automatically initiate parallel re-replication, prioritizing the most under-replicated data to restore redundancy across the cluster.

10.4 Sum-up

Part 1: Google File System (GFS)

- **Master-Chunkserver Communication (Heartbeats):**
  - GFS uses passive **Heartbeat** messages instead of synchronous polling to prevent the single master from becoming a performance bottleneck[cite: 3, 4].
  - *Information Transferred:* Chunkservers send **State reports** about their chunks[cite: 6]. The master responds with **Instructions** (e.g., data replication, cleanup, or sending chunk leases)[cite: 7, 166].
  - This allows the master to monitor chunkserver health, detect failures, and update chunk records passively[cite: 5, 8].
- **Advantages of 64 MB Chunk Size:**
  - It reduces client-master interactions because clients only need to ask for a chunk's location once[cite: 9, 312].
  - It minimizes network overhead by allowing clients to maintain a persistent TCP connection to the chunkserver for consecutive operations[cite: 12, 13, 315].
  - It reduces the total size of metadata, allowing the master to keep all metadata in memory for faster performance[cite: 14, 317, 334].
- **Client-Master Read Optimization:**
  - GFS uses **Metadata Caching**[cite: 15, 278]. Clients ask the master for the chunk handle and replica locations, and then cache this information using the filename and chunk index as a key[cite: 17, 18, 302].
  - Clients never read or write actual data through the master; they interact directly with chunkservers[cite: 16, 296].
- **Chunk Location Mapping:**
  - At startup, the master does not have a persistent record of chunk locations; it polls all chunkservers to discover their chunks[cite: 20, 21, 332, 344].
  - After startup, the master updates its mapping because it controls all new chunk placements and monitors chunkservers via Heartbeats[cite: 21, 345].
- **The Operation Log & Checkpoints:**
  - The Operation Log is the only persistent record of critical metadata and defines the logical timeline of concurrent operations[cite: 22, 329, 353].
  - Changes are committed only after the log record is batched and flushed to local and remote disks[cite: 24, 357, 358].
  - The master periodically compacts the log into memory-mappable **Checkpoints** to minimize startup and recovery times by replaying only the log records written after the latest checkpoint[cite: 25, 361, 376].
- **File Region States after Mutation:**
  - **Consistent:** All clients see the exact same data from all replicas[cite: 28, 393].
  - **Defined:** The region is consistent, and clients see the mutation in its entirety without interference from concurrent writes[cite: 29, 30, 394, 395].
  - **Undefined:** The region is consistent (all clients see the same data), but it contains mixed fragments from concurrent writes[cite: 33, 396, 397].

  - **Inconsistent:** A mutation fails; different clients read different data from different replicas at different times[cite: 31, 32, 397].

- **Consistency Mechanics & Mutation Order:**
  - The master grants a **lease** to a primary replica[cite: 41, 441]. The primary replica orders mutations by assigning consecutive serial numbers[cite: 34, 42, 477].
  - Secondary replicas must apply mutations in the exact order assigned by the primary[cite: 43, 480].
  - Stale replicas are detected using **chunk version numbers**, which are increased each time the master grants a new lease[cite: 35, 45, 649, 650]. Stale replicas are excluded from client operations and garbage collected[cite: 37, 407, 656].
  - Data integrity is verified independently by chunkservers using 32-bit checksums for every 64 KB block within a chunk[cite: 39, 181, 701].
- **Data Transfer Strategy (Network Optimization):**
  - GFS decouples control flow from data flow[cite: 46, 473].
  - Data is pushed linearly along a chain of chunkservers (not a tree) to utilize each machine's full outbound bandwidth[cite: 47, 494, 496].
  - Machines forward data to the closest remaining chunkserver based on IP distance[cite: 48, 498].
  - Latency is minimized through **pipelining**: a chunkserver immediately begins forwarding data over TCP as soon as it receives a portion of it[cite: 49, 504, 505].
- **Relaxed Consistency (Byte-wise Identity):**
  - Replicas do not need to be byte-wise identical because GFS guarantees that data is written "at least once" as an atomic unit[cite: 50, 52, 528, 529].
  - Failures and client retries can create duplicate data or padding across replicas[cite: 51, 403, 527].
  - GFS applications handle this by design using checkpoints, verifying checksums to discard padding, or filtering duplicates with unique record identifiers[cite: 53, 54, 55, 419, 431, 432].
- **Replica Placement Policy:**
  - *Goals:* Maximize data reliability, availability, and network bandwidth utilization[cite: 56, 585].
  - *Strategy:* Spread replicas across different racks to survive complete rack failures (e.g., switch or power issues) and exploit aggregate read bandwidth from multiple racks[cite: 58, 59, 60, 587, 588].
- **Chunk Creation & Re-replication:**
  - *Placement of New Chunks:* The master places new chunks on chunkservers with below-average disk utilization, limits "recent" creations on a single node to avoid heavy write traffic, and spreads replicas across racks[cite: 63, 64, 66, 593, 594, 596]. *Re-replication triggers:* Initiated when the replica count falls below a specified goal (default is 3) due to chunkserver unavailability, disk errors, data corruption, or an increased replication goal[cite: 68, 69, 70, 597, 598].
- **High Availability & Shadow Masters:**
  - Master reliability is achieved by replicating the operation log and checkpoints on multiple remote machines[cite: 71, 682].
  - GFS prefers **Shadow Masters** over mirror masters to maintain simplicity[cite: 76].
  - Shadow masters update themselves passively by reading a replica of the operation log[cite: 77, 692]. They may lag behind the primary master by fractions of a second, but they safely provide

read-only availability for metadata while actual file content is read directly from chunkservers[cite: 77, 78, 79, 687, 690].

Part 2: Hadoop Distributed File System (HDFS)

- **Metadata Concepts (Image, Checkpoint, Journal):**
  - **Image:** The complete in-memory state of the file system metadata, containing inodes attributes and file-to-block mappings[cite: 81, 1065].
  - **Checkpoint:** The persistent, read-only record of the namespace image stored in the local file system[cite: 82, 1066, 1137].
  - **Journal:** A persistent, write-ahead commit log that records every metadata modification transaction before committing[cite: 83, 1067, 1138].
- **NameNode-DataNode Communication:**
  - DataNodes send **Heartbeats** every 3 seconds to confirm they are active[cite: 1092, 1093].
  - Heartbeats carry metrics on total storage capacity, storage in use, and data transfers in progress[cite: 85, 1095].
  - The NameNode never directly calls DataNodes; it delivers commands by piggybacking instructions onto Heartbeat replies[cite: 86, 87, 1097].
- **Startup Handshake & Registration:**
  - During startup, a DataNode performs a **handshake** with the NameNode to verify the cluster's **namespace ID** and the **software version**[cite: 89, 90, 1076, 1077]. A mismatch causes the DataNode to shut down to prevent data corruption[cite: 91, 1078, 1082].
  - After the handshake, the DataNode registers and persistently stores a unique **storage ID**, which identifies the node even if its IP address or port changes[cite: 92, 93, 1084, 1085].
- **Flexible Replication Factors:**
  - HDFS can handle different replication factors for different files via an explicit API (the default factor is 3)[cite: 94, 95, 96, 1057, 1133, 1134].
  - Critical or heavily accessed files are assigned a higher replication factor to increase fault tolerance and read bandwidth[cite: 97, 1134].
- **Data Corruption & Verification:**
  - When writing, the client computes checksum sequences and sends them to the DataNode, which stores them in a separate metadata file[cite: 99, 1259, 1260].
  - At read time, block data and checksums are shipped to the client to verify integrity[cite: 100, 1261, 1262]. Mismatches trigger a notification to the NameNode, and the client fetches another replica[cite: 101, 1263].
  - Corruption is also verified independently in the background: each DataNode runs a **Block Scanner** that periodically verifies stored checksums against block data[cite: 102, 103, 1350].
- **Metadata Storage Practices & CheckpointNode:**
  - Recommended practices include writing metadata to multiple directories across separate physical volumes and placing one copy on a remote NFS server to protect against single-node failures[cite: 104, 105, 1145, 1146].
  - Checkpoints should be created daily because a massive journal increases corruption risks and NameNode restart times[cite: 106, 1167, 1169].

  - The **CheckpointNode** periodically downloads the checkpoint and journal, merges them locally, and uploads the new checkpoint back to the NameNode, allowing the NameNode to truncate its journal[cite: 107, 108, 1158, 1160, 1164].
- **File Access Constraints (Random Writes):**
  - HDFS does **not** allow random writes at arbitrary offsets[cite: 110].
  - It is a write-once-read-many architecture[cite: 111]. Once a file is closed, written bytes cannot be modified[cite: 111, 1207]. The only allowed write operation is **appending** data at the end of the file[cite: 112, 1207].
- **File Write Pipeline & Concurrency Rules:**
  - The NameNode provides a block ID and a list of target DataNodes organized as a pipeline to minimize total network distance[cite: 115, 116, 1218, 1219].
  - Client writes are first buffered locally, split into packets (typically 64 KB), and pushed into the pipeline using a sliding window for outstanding packets[cite: 118, 119, 1221, 1222, 1224].
  - *Concurrency:* HDFS enforces a **single-writer, multiple-reader model**[cite: 131, 1208]. The writer is granted an exclusive **lease**[cite: 123, 1209]. No other client can write to the file simultaneously[cite: 124, 1209].
- **File Read Mechanics & Concurrent Operations:**
  - The HDFS client requests block locations from the NameNode[cite: 127, 1108]. Locations are sorted by network distance relative to the reader[cite: 128, 1265].
  - The client contacts the closest DataNode directly; if it is unavailable or the data is corrupt, it tries the next replica sequentially[cite: 129, 130, 1266, 1267].
  - Multiple clients can read a file concurrently while a write operation is active[cite: 131, 132, 1217].
  - If a file is open for writing, the length of the unclosed block is unknown to the NameNode, so the reader asks a replica directly for its latest length before reading[cite: 133, 134, 1270, 1271].
- **Default Block Placement Policy:**
  - The first replica goes on the node hosting the writer application[cite: 136, 138, 1299].
  - The second and third replicas are placed on two separate nodes inside a different, common rack[cite: 136, 138, 1299].
  - This strategy provides a tradeoff: it minimizes inter-rack write traffic (improving write speed) while preserving strong reliability against rack switch failures and reducing aggregate network bandwidth during reads since data spans only two racks[cite: 140, 141, 142, 1307, 1308, 1309].
- **Cluster Disk Rebalancing:**
  - Initial block placement ignores disk utilization to prevent clustering heavy reference data on newly added, empty nodes[cite: 143, 1333, 1334].
  - Balance is achieved using an administrative tool called the **Balancer**[cite: 144, 1338]. It iteratively moves replicas from highly utilized DataNodes to less utilized ones until individual node utilization falls within a specified threshold of the cluster average[cite: 145, 146, 1340, 1342].
  - The Balancer guarantees data availability by never reducing the number of replicas or racks, and optimizes traffic by prioritizing intra-rack copying[cite: 147, 148, 1344, 1345].

Part 3: Deep Comparison (GFS vs. HDFS)

Feature	Google File System (GFS)	Hadoop Distributed File System (HDFS)
Storage Units	Chunks of fixed 64 MB size stored as standard Linux files[cite: 153, 266, 267].	Blocks of 64–128 MB size stored as native files[cite: 154, 1057, 1072].
Client-Node Read	Client requests chunk locations, caches them, and interacts directly with chunkservers[cite: 149, 150, 302].	Client asks the NameNode for block locations and reads from the closest DataNode[cite: 151, 152, 1060].
Modification Flow	Client pushes data to all replicas in any order, then primary replica sequences control[cite: 156, 471, 475].	Client writes data to the first DataNode, which forwards it sequentially to the next nodes in a pipeline[cite: 157, 1062, 1111].
Storage Mapping	Reconstructed from scratch during startup via chunkserver reports[cite: 160, 332].	Mapping is transient; built from scratch via DataNode block reports[cite: 160, 1070].
Node Validation	Relaxed background checks; nodes report a subset of chunks in regular heartbeats[cite: 164, 625].	Strict startup handshake verifying namespace ID and software version; full block reports every hour[cite: 162, 163, 1076, 1091].
Placement Strategy	Highly flexible; master evaluates disk space and dynamically shifts placements[cite: 169, 592].	Fixed rack-aware rule: local node for writer, two different nodes on a separate rack for replicas 2 and 3[cite: 170, 1299].
Cluster Balancing	Automated continuously in the background by the master process[cite: 171, 608].	Requires an external administrative program (*Balancer*) executed manually[cite: 171, 1338, 1341].
Stale Replicas	Tracked precisely using incrementing chunk version numbers[cite: 172, 649, 650].	Tracked precisely using incrementing *generation stamps* on data files[cite: 172, 1073].
Excess Cleanup	Master removes replicas on nodes with below-average free space[cite: 612].	Prioritizes preserving rack spread, then removes from nodes with the least free space[cite: 173, 1318].
Concurrent Writes	Supported natively via **Atomic Record Append** across multiple writers[cite: 178, 259].	Strictly prohibited; implements a strict **single-writer model** via exclusive leases[cite: 179, 1208, 1209].
Arbitrary Overwrites	Supported at any position, though unoptimized and slow[cite: 247].	Blocked completely; previously written bytes are completely immutable[cite: 111, 1207].
High Availability	Provided by **Shadow Masters**, which offer read-only replication lagging by seconds[cite: 183, 687, 688].	Provided by **BackupNodes**, which hold a fully synchronized, active namespace image[cite: 183, 1171, 1172].

Table 10.1: Key Similarities and Differences between GFS and HDFS[cite: 158, 159, 174, 175, 176, 177, 182, 184].

11. FC5 - Cloud DB: Dynamo & BigTable

11.1 Task 1 - Google Big Table

11.1.1 Q1.1 The Google SSTable file format is used to store Bigtable data. What does an SSTable internally contain? What are the core steps of an SSTable lookup?

An SSTable is a persistent, ordered, immutable map from keys to values, where both keys and values are arbitrary byte strings. Internally, it contains:

- a sequence of data blocks
- a block index, stored at the end of the SSTable, used to locate the blocks.

Typically, each block is about 64 KB, although this is configurable. When the SSTable is opened, its block index is loaded into memory. A lookup then works in these main steps:

1. Binary search the in-memory block index to find the block that should contain the key
2. Read the corresponding block from disk
3. Find the key/value inside that block

This means a lookup can be done with a single disk seek. Optionally, the whole SSTable can be memory-mapped, which allows reads without touching the disk.

11.1.2 Q1.2 BigTable heavily relies on the Chubby lock service. Explain what Chubby is, and why and how it is used by BigTable.

Chubby is a highly-available and persistent distributed lock service that provides a namespace of directories and small files that can be used as a lock, and reads and writes to these files are atomic. Bigtable fundamentally relies on Chubby for central coordination, and its overall operation is so dependent on this service that if

Chubby becomes unavailable for an extended period of time, Bigtable becomes unavailable.
BigTable nodes maintain a connection (session) with Chubby. If a node loses this connection, its session expires.
BigTable uses Chubby for 5 main tasks:

- **Choosing the Master**: BigTable ensures there is only one Master at a time. The Master does this by taking a unique lock in Chubby;
- **Finding the Data (Bootstrap)**: BigTable needs to know where the data starts. It reads a specific file in Chubby to find the location of the Root Tablet;
- **Discovering Servers**: When a new Tablet Server starts, it creates a unique file in Chubby and locks it. The Master watches this directory to see which servers are active;
- **Handling Dead Servers**: If a Tablet Server loses its lock, the Master deletes that server's file. This stops the server from working permanently;
- **Storing Metadata**: BigTable uses Chubby as a safe place to store schema information (table structures) and access control lists (security permissions).

11.1.3 Q1.3 How is a BigTable split among multiple files?

A BigTable is dynamically partitioned into smaller logical units based on its row keys. Initially, a table consists of just one of these units, but as a table grows, it is automatically split into multiple tablets, each approximately 100-200 MB in size by default.
To manage the physical storage of these tablets, the persistent state of a tablet is stored in GFS. Rather than keeping everything in one massive file, the system divides the data using a specific format where older updates are stored in a sequence of SSTables. Ultimately the Google SSTable file format is used internally to store BigTable data and these physical files are further subdivided internally so that each SSTable contains a sequence of block (typically each block is 64KB in size, but this is configurable) which allows the system to efficiently read small portions of data without having to scan the entire file.

11.1.4 Q1.4 What are the responsibilities of the BigTable Master server and of the Tablet servers?

In the Bigtable architecture, the workload is strictly divided between a single master and multiple tablet servers. The master acts as the central coordinator for metadata and administrative tasks. The master is responsible for assigning tablets to tablet servers, detecting the addition and expiration of tablet servers, balancing tablet-server load and garbage collection of files in GFS. Furthermore, it handles schema changes such as table and column family creations, and during system failures, it is responsible for detecting when a tablet server is no longer serving its tablets, and for reassigning those tablets as soon as possible.
Conversely, the tablet servers do the actual heavy lifting of storing and serving user data. Each tablet server manages a set of tablets, and it directly handles read and write requests to the tablets that it has loaded, and also splits tablets that have grown too large. To ensure high performance and keep the master lightly loaded, clients communicate directly with tablet servers for reads and writes, meaning that actual client data never moves through the master server.

11.1.5 Q1.5 How are the Tablet locations stored and handled?

BigTable tracks tablet locations using a 3-level hierarchy (similar to a B^+-tree). This structure never grows larger because the Root Tablet is never split.Level 1 (Chubby File): Contains the location of the Root

Tablet.Level 2 (Root Tablet): Contains the locations of all other METADATA tablets.Level 3 (METADATA Tablets): Contains the actual locations of the User Data tablets.Capacity: Each METADATA tablet is limited to about 128MB. Thanks to this 3-level structure, the system can store a massive amount of data.
On the client side, the library keeps a cache of tablet locations to avoid repeated lookups; if the client does not know where a tablet is, or if its cached information is wrong, it looks it up by moving up the hierarchy: if the cache is empty, it takes about three network requests (including one to Chubby), while if the cache is outdated, it may take up to six requests; to improve performance, the client also prefetches data by reading information for multiple tablets at once when accessing the METADATA table.

11.1.6 Q1.6 How does the master node discover Tablet servers? And how does it check if a Tablet server is no longer serving the assigned Tablets?

Bigtable uses Chubby to keep track of which tablet servers are running, so when a tablet server starts, it creates a unique file in Chubby and locks it so no one else can use it, and then the master watches a specific directory in Chubby to see which servers are active. When the master starts, it does the following: it gets a special lock in Chubby, so only one master can run at a time, it checks the servers directory to find all active tablet servers, it contacts each server to see which tablets they are currently serving and then it scans the METADATA table to get the full list of tablets, and marks any unassigned ones as needing assignment. Then, the master regularly checks if tablet servers are still alive: it asks each server whether it still holds its lock in Chubby and if a server says it lost the lock, or the master cannot reach it for a while, the master tries to take over that server's lock; however, if the master successfully takes the lock, it means the server is either dead or disconnected from Chubby, so the master deletes the server's file so it cannot come back and serve data incorrectly and all tablets that were assigned to that server are marked as unassigned and will be reassigned. Tablet servers also monitor themselves: if a server loses its lock it stops serving tablets immediately; a server tries to get the lock back as long as its file still exists; if the file has been deleted, the server shuts itself down completely. Overall, this system ensures that only healthy servers serve data, and failures are quickly detected and handled.

11.1.7 Q1.7 What are the actions performed by the Big Table Master if it discovers a Tablet server fails?

If the master determines that a tablet server has failed or can no longer safely serve tablets, it:

1. acquires the server's lock/file in Chubby, if possible;
2. deletes the server file, ensuring that server can never serve again;
3. moves all tablets previously assigned to that server into the unassigned set;
4. later reassigns those tablets to other tablet servers.

This is how Bigtable recovers ownership of tablets after server failure.

11.1.8 Q1.8 What does the Master do if the connection with Chubby is disrupted? Does this failure impact the assignment of Tablets to Tablet servers?

If the master's Chubby session expires, the master kills itself. This is done to avoid unsafe behavior when coordination with Chubby is lost. However, this does not change tablet assignments. The paper explicitly says that master failures do not change the assignment of tablets to tablet servers. So the cluster can continue with the current assignments; what is affected is the master's ability to coordinate further changes until a new master comes up.

11.1.9 Q1.9 What are the steps executed by the BigTable Master at startup?

1. Grab a unique master lock in Chubby: This prevents multiple master instances from running at the same time.
2. Scan the servers directory in Chubby: This allows the master to discover all the live tablet servers.
3. Communicate with every live tablet server: The master queries each server to learn which tablets are already assigned to them.
4. Scan the METADATA table: The master scans this table to get a complete list of all existing tablets. If it finds any tablets that weren't assigned during step 3, it adds them to the pool of unassigned tablets, making them available for assignment.

11.1.10 Q1.10 When does the set of existing tablets change?

The set of existing tablets in Bigtable only changes under the following four circumstances:

- **Table Creation**: When a new table is first established;
- **Table Deletion**: When an existing table is removed from the system;
- **Tablet Merging**: When two existing tablets are combined to form a single, larger tablet;
- **Tablet Splitting**: When an existing tablet is divided into two smaller tablets.

11.1.11 Q1.11 How do the read and write operations at a Tablet server work? How a Tablet server recovers a Tablet?.

When a write arrives, the tablet server:

1. checks that the request is well formed;
2. checks that the sender is authorized;
3. appends the mutation to the commit log;
4. uses group commit to improve throughput for many small writes;
5. inserts the mutation into the memtable.

When a read arrives, the tablet server:

1. checks that the request is well formed;
2. checks authorization;
3. executes the read over a merged view of the memtable and the tablet's SSTables.

This is efficient because both SSTables and memtables are sorted.
To recover a tablet, a tablet server:

1. reads the tablet's metadata from the METADATA table;
2. obtains the list of SSTables belonging to the tablet;
3. obtains the redo points into commit logs;
4. loads SSTable indices into memory;
5. reconstructs the memtable by replaying updates committed since those redo points.

So recovery combines the immutable SSTables with replay of the relevant logged updates.

11.1.12 Q1.12 BigTable maintains a single commit log for each tablet server. How is it implemented and handled to enable an efficient recovery when the tablets of a failed tablet server are reassigned to many tablet servers?

Bigtable uses one commit log per tablet server, not one per tablet. Mutations for different tablets are co-mingled in the same physical log file. This improves normal operation because:

- it avoids many concurrently written log files in GFS;
- it reduces disk seeks;
- it improves group commit efficiency.

But this makes recovery harder: after a tablet server fails, many different tablet servers may each need just part of that failed server's log. Reading the full log independently on every recovery server would be very wasteful.

To solve this, Bigtable:

1. sorts the commit-log entries by ⟨table, row name, log sequence number⟩;
2. after sorting, all mutations for a given tablet become contiguous;
3. this allows each recovery server to read only the relevant portion efficiently, with one disk seek plus sequential read;
4. to parallelize recovery, the log is partitioned into 64 MB segments, and each segment is sorted in parallel on; different tablet servers
5. this sorting is coordinated by the master.

In addition, each tablet server uses two log-writing threads, each writing to its own log file. If one log path becomes slow, Bigtable switches to the other. Log entries contain sequence numbers so duplicate entries caused by switching can be removed during recovery. Also, when a tablet is moved, the source tablet server performs minor compactions before unloading it, which reduces the amount of log replay needed later.

11.2 Task 2 - Dynamo DB

11.2.1 Q2.1 Why is partitioning introduced, and how is it implemented?

Partitioning is introduced because Dynamo is designed to easily scale incrementally and this requires a mechanism to dynamically partition the data across many nodes without major reshuffling every time the system grows.

11.2.2 Q2.2 How are data assigned to storage nodes? How is the subset of data a node is responsible for determined?

Dynamo uses consistent hashing to distribute data: the output range of a hash function is treated as a fixed circular space or "ring", where each node (server) and each data item (key) is placed at a random position on this ring; to find where a key belongs, you move clockwise on the ring until you reach the first node, the one that stores the data; this approach makes it easy to add or remove nodes with minimal data movement. However, basic consistent hashing has some issues: the nodes might end up unevenly spaced, causing some nodes to store much more data than others, and it does not consider that some machines are more powerful than others; to fix this, Dynamo uses virtual nodes: instead of each physical machine having just one position on the ring, it gets multiple positions and each of these positions acts like a small "virtual node". The advantages of virtual nodes are that: if a node becomes unavailable its load is spread more evenly across all machines;

when a new node is added it accepts a roughly equivalent amount of load from the other nodes; more powerful machines can be assigned more virtual nodes, so they handle more data.

11.2.3 Q2.3 What are the concerns in using consistent hashing, and how are they resolved in Dynamo?

The primary challenges with a basic consistent hashing algorithm are its tendency to distribute data and load unevenly across nodes and its inability to account for performance differences between those nodes. The random placement of a single token per node on the ring can create "hot spots," where some nodes are responsible for a disproportionately large amount of data.

Dynamo addresses these issues by introducing the concept of "virtual nodes." Instead of assigning just one position on the ring to a physical server, each server is responsible for multiple virtual nodes, each at a different point on the ring. This approach leads to a more uniform and balanced distribution of data. Furthermore, it allows Dynamo to account for system heterogeneity by assigning more virtual nodes to more powerful servers, ensuring that a machine's load is proportional to its capacity. This use of virtual nodes ensures that when a server is added or removed, the load is smoothly redistributed across many other nodes in the system rather than just its immediate neighbors.

11.2.4 Q2.4 Which data replication strategy is used in Dynamo? How can it be guaranteed that the N replicas of data are stored on distinct physical nodes?

Dynamo uses an optimistic replication strategy, which results in an eventually consistent system. Instead of enforcing strict, synchronous updates where a write must succeed on all replicas before returning, Dynamo prioritizes being "always writeable." Changes are propagated to replicas in the background, which ensures high availability for write operations even during server or network failures. This design choice means that conflicts can occur if different replicas are updated concurrently. To handle this, Dynamo pushes the complexity of conflict resolution to the read operation, allowing the application to decide how to merge different versions of the data.

Dynamo guarantees that N replicas are stored on distinct physical nodes. However, it does highlight Symmetry ("Every node in Dynamo should have the same set of responsibilities") and Decentralization as key design principles. These principles, combined with a partitioning algorithm (like the one discussed previously), are fundamental to ensuring that replicas are distributed across different physical hosts to avoid a single point of failure.

11.2.5 Q2.5 Why is data versioning introduced, and how is it implemented? When are two changes considered to be in conflict and require reconciliation?

Dynamo provides eventual consistency, In order to provide this kind of guarantee Dynamo uses vector clocks to capture causality between different versions of the same object. A vector clock is effectively a list of (node, counter) pairs. One vector clock is associated with every version of every object. One can determine whether two versions of an object are on parallel branches or have a causal ordering, by examining their vector clocks. If the counters on the first object's clock are less-than-or-equal to all of the nodes in the second clock, then the first is an ancestor of the second and can be forgotten. Otherwise, the two changes are considered to be in conflict and require reconciliation. Version conflicts may happen, in the presence of failures combined with concurrent updates, resulting in conflicting versions of an object. In these cases, the system cannot reconcile

the multiple versions of the same object and the client must perform the reconciliation in order to collapse multiple branches of data evolution back into one (semantic reconciliation). A typical example of a collapse operation is “merging” different versions of a customer’s shopping cart. Using this reconciliation mechanism, an “add to cart” operation is never lost.

11.2.6 Q2.6 What are the possible actions performed by a node that receives a put request? When the write operation is considered successful?

When a node receives a put() request, it acts as the coordinator for that operation. The coordinator executes the following steps:

1. **Generates a New Version**: It creates a new vector clock for the object being written. This version information is crucial for capturing the object’s history and handling future conflicts;
2. **Writes Locally**: The coordinator writes the new version of the object to its own local storage;
3. **Forwards to Replicas**: It then sends the new object versionand its new vector clock to the N highest-ranked reachable nodes responsible for that key (this group of nodes is called the preference list);
4. **Waits for Responses**: The coordinator waits to receive acknowledgment from a subset of these replica nodes.

11.2.7 Q2.7 What is a hinted replica? How does hinted handoff work?

If a node is temporarily down or unreachable during a write operation then a replica that would normally have lived on that node will now be sent to another node. This is done to maintain the desired availability and durability guarantees. The replica sent to the another node will have a hint in its metadata that suggests which node was the intended recipient of the replica. Nodes that receive hinted replicas will keep them in a separate local database that is scanned periodically.

Using hinted handoff, Dynamo ensures that the read and write operations are not failed due to temporary node or network failures. Applications that need the highest level of availability can set W to 1, which ensures that a write is accepted as long as a single node in the system has durably written the key it to its local store. Thus, the write request is only rejected if all nodes in the system are unavailable. However, in practice, most Amazon services in production set a higher W to meet the desired level of durability.

11.2.8 Q2.8 How does the "T random tokens per node and partition by token value" partitioning strategy work? What are its limitations/issues?

“T random tokens per node and partition by token value strategy” assignes each node several random points, called tokens, on a circular hash ring. A node became responsible for any data falling in the ranges between its tokens and those of its neighbors. Because these points were chosen randomly, the resulting data ranges were often uneven in size, which made it difficult to distribute the workload perfectly.

The real trouble began when Amazon tried to scale the system by adding new nodes. To join the ring, a new node had to "steal" ranges from existing members, forcing those members to scan their entire local databases to find the specific keys that needed to be moved. Since these scans were heavy operations, they were run at a very low priority to avoid slowing down actual customer requests. Consequently, adding a single node could take nearly a day during busy seasons. Furthermore, any change in the ring required nodes to recalculate their Merkle trees, which are complex structures used to keep data synchronized.

Ultimately, the fundamental flaw of this strategy was that data partitioning and node placement were "inter-

twined". You could not simply add a server to handle more traffic without also forcing the system to change how it divided up the data. This lack of independence made the system difficult to manage and archive, eventually leading Amazon to move toward more flexible strategies that decoupled the data from the physical nodes.

11.2.9 Q2.9 Discuss the differences between the "T random tokens per node and equal-sized partitions" and "Q/S tokens per node, equal-sized partitions" partitioning strategies.

Both Strategies divides the hash space into Q equally sized partitions. But they act different in :

- **Token assignment**: in Strategy 2 Each node is assigned T tokens at random positions. Instead in strategy 3 Each node is assigned Q/S tokens, where S is the number of nodes in the system;
- **Load distribution Efficiency**: strategy 3 achieves the best load balancing efficiency and strategy 2 has the worst load balancing efficiency.

11.3 Sum-up

11.3.1 Google BigTable Architecture

- **Data Model**: A sparse, distributed, persistent multi-dimensional sorted map[cite: 1666]. It is indexed by a row key, column key, and a timestamp; values are uninterpreted byte arrays[cite: 1667, 1668]. Reads/writes under a single row key are atomic[cite: 1687, 1688].
- **SSTable Internals**: Persistent, ordered, immutable maps from byte strings to byte strings[cite: 1764]. They internally contain a sequence of data blocks (usually 64 KB) and a block index stored at the end of the file[cite: 1768, 1769].
- **SSTable Lookup Steps**:
  1. Perform a binary search on the in-memory block index to find the correct block[cite: 1771].
  2. Read the corresponding data block from disk via a single seek[cite: 1771].
  3. Find the target key/value pair inside that block[cite: 1767, 1771].
  4. *Optional*: If the SSTable is fully memory-mapped, lookups require no disk access[cite: 1772].
- **Chubby Lock Service**: A highly-available distributed lock service based on Paxos[cite: 1773, 1776]. BigTable completely relies on it [cite: 1773, 3130]; if Chubby is down, BigTable becomes unavailable[cite: 1788]. It handles 5 major tasks[cite: 3131]:
  - *Master Election*: Ensures at most one active master exists at a time[cite: 1784].
  - *Data Bootstrap*: Stores the bootstrap location of the Root Tablet[cite: 1785].
  - *Server Discovery*: Discovers live tablet servers when they register and lock a unique file[cite: 1832, 1833].
  - *Handling Failures*: Finalizes server deaths by allowing the master to take over and delete their server files[cite: 1786, 1842].
  - *Metadata Storage*: Stores table schema info and access control lists[cite: 1787].
- **Master vs. Tablet Server Responsibilities**:
  - *Master Server*: Acts as the coordinator[cite: 3145]. Assigns tablets, detects live/dead tablet servers, balances load, collects garbage files in GFS, and handles schema updates[cite: 1797, 1798].
  - *Tablet Servers*: Manage a set of 10 to 1,000 tablets[cite: 1799]. They handle direct read and write requests and split tablets that grow too large[cite: 1800]. Clients talk directly to them, so data

never passes through the master[cite: 1801].

- **Tablet Partitioning and Lifecycle**: Tables start as a single tablet and automatically split into multiple tablets (100–200 MB by default) as they grow[cite: 1805]. The set of existing tablets changes only during table creation, table deletion, tablet merging, or tablet splitting[cite: 1855].
- **Tablet Location (3-Level Hierarchy)**: Similar to a B^+-tree[cite: 1807]. Level 1 is a file in Chubby pointing to the Root Tablet[cite: 1814]. Level 2 is the Root Tablet, which contains locations of all other METADATA tablets[cite: 1815]. Level 3 consists of METADATA tablets containing the locations of user tablets[cite: 1816]. The Root tablet is never split to bound the hierarchy to 3 levels[cite: 1817]. Clients cache these locations and recursively look up the tree on a cache miss[cite: 1822].
- **Master Startup Steps**:
  1. Grabs a unique master lock in Chubby to prevent concurrent master instances[cite: 1847].
  2. Scans the Chubby servers directory to find all live tablet servers[cite: 1848].
  3. Communicates with every live tablet server to discover currently assigned tablets[cite: 1849].
  4. Scans the METADATA table to find unassigned tablets and places them in the unassigned pool[cite: 1850, 1851].
- **Chubby Disruption**: If the master's Chubby session expires, the master kills itself to maintain coordination safety[cite: 1844]. However, master failures do not impact or alter the existing assignment of tablets to tablet servers[cite: 1845].
- **Tablet Server Operations**:
  - *Writes*: Checks for well-formedness and authorization[cite: 1876]. Appends the mutation to a single shared commit log per tablet server (improves disk seek efficiency and group commit)[cite: 1878, 1947]. Once committed, the write is inserted into the sorted in-memory memtable[cite: 1879].
  - *Reads*: Checks well-formedness and authorization [cite: 1880], then executes the read over an efficiently merged view of the sorted memtable and SSTables[cite: 1881, 1882].
  - *Recovery*: Reads the tablet's metadata from the METADATA table to get the list of SSTables and redo points[cite: 1874]. It loads the SSTable indices into memory and reconstructs the memtable by replaying updates from the logs[cite: 1875].
- **Commit Log Optimization for Recovery**: Bigtable co-mingles mutations of multiple tablets into a single physical log file per server[cite: 1947]. To avoid duplicate log reads during recovery when tablets are reassigned to many servers [cite: 1953, 1954], the log entries are sorted by ⟨table, row name, log sequence number⟩[cite: 1955]. This groups mutations contiguously, allowing efficient single-seek sequential recovery reads[cite: 1956].

11.3.2 Amazon Dynamo DB Architecture

- **Partitioning Motivation**: Introduced to achieve incremental scalability by dynamically spreading data across storage nodes with minimal manual rebalancing or operational intervention[cite: 2343, 2543, 2544].
- **Data Assignment via Consistent Hashing**: The hash output range is treated as a circular ring[cite: 2546]. Nodes and keys are mapped onto this ring[cite: 2547, 2548]. To find the storage destination for a key, the system walks clockwise from the key's hash position to find the first node with a larger position[cite: 2548].

- **Consistent Hashing Issues and Virtual Nodes**: Basic consistent hashing leads to non-uniform data distribution (load imbalance/hotspots) and ignores hardware heterogeneity[cite: 2552, 2553]. Dynamo resolves this via *virtual nodes*[cite: 2554, 2555]. Each physical server is assigned multiple tokens (positions) on the ring[cite: 2557]. This disperses load evenly on node failure [cite: 2561], smoothly integrates new servers [cite: 2562], and assigns more positions to higher-capacity hardware[cite: 2563].
- **Replication Strategy**: Optimistic and asynchronous replication is used to build an eventually consistent, "always-writeable" store[cite: 2341, 2434, 2441]. Writes return before propagating to all replicas[cite: 2579, 2580]. Replicas are stored at N distinct physical hosts[cite: 2565]. The coordinator node maps the key locally and replicates it across the $N-1$ clockwise physical successors on the ring [cite: 2569], skipping duplicate physical nodes if virtual nodes overlap[cite: 2576, 2577].
- **Data Versioning and Conflicts**: Introduced to maintain write availability under network partitions and server outages[cite: 2581, 2583]. Dynamo treats modifications as new, immutable object versions[cite: 2588]. It implements *vector clocks* (a list of $\langle$node, counter$\rangle$ pairs) to capture causality[cite: 2598, 2599].
  - *Causal ordering / Ancestry*: If version A's clock counters are less-than-or-equal to all node counters in version B's clock, version B dominates A, and A can be forgotten[cite: 2602].
  - *Conflict*: If neither clock dominates, the changes are concurrent and causally unrelated[cite: 2635, 2636]. They require client-side semantic reconciliation during the next read operation (e.g., merging cart versions) to collapse the branches back into one[cite: 2591, 2592, 2637].
- **Put Request Actions and Success Criteria**: A node receiving a `put()` request acts as the coordinator[cite: 2655]. It generates a new vector clock version stamp [cite: 2669], writes the new data locally [cite: 2669], and forwards the update with the vector clock to the N highest-ranked reachable replica nodes in the preference list[cite: 2672]. The write is considered successful as soon as at least $W-1$ replica nodes respond with an acknowledgment[cite: 2673].
- **Hinted Replica and Hinted Handoff**: If a primary node is down during a write, the coordinator sends the replica to a healthy downstream node lower on the ring[cite: 2680, 2682]. This replica contains a *hint* in its metadata indicating its intended destination node[cite: 2683]. The receiving node stores it in a separate database and periodically scans it[cite: 2684]. Upon detecting that the original node has recovered, it delivers the hinted replica back and can safely delete it locally[cite: 2685]. This guarantees write availability amidst temporary failures[cite: 2686].
- **Partitioning Strategy Evolution**:
  - *Strategy 1 (T random tokens per node, partition by token value)*: Initial approach[cite: 2912]. Tokens randomly carve variable-sized ranges[cite: 2913, 2916]. Issues: Data partitioning and placement are intertwined[cite: 2927]. Bootstrapping a new node requires existing nodes to perform highly resource-intensive local database scans to gather key ranges [cite: 2919, 2921], slowing bootstrap times to nearly a day[cite: 2923]. Ring changes also force complex Merkle tree recalculations [cite: 2924], and random ranges make data archival highly inefficient[cite: 2925, 2926].
  - *Strategy 2 (T random tokens per node, equal-sized partitions)*: Divided hash space into Q equally-sized partitions to decouple partitioning from placement [cite: 2931, 2936], but suffered from the worst load balancing efficiency[cite: 2960]. Used only as an interim setup during migration[cite:

2961].

- *Strategy 3 (Q/S tokens per node, equal-sized partitions)*: Divides the hash ring into Q fixed, equally-sized partitions, and each of the S physical nodes is assigned Q/S tokens[cite: 2937, 2938]. It delivers the best load balancing efficiency and drastically reduces membership metadata sizes[cite: 2960, 2961]. Because partition ranges are completely fixed, they are stored as distinct physical files[cite: 2963]. This allows whole partitions to be transferred as units during bootstrap or recovery without expensive database scans, resulting in faster bootstrapping, simpler deployment, and clean data archival[cite: 2963, 2964, 2965].